

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Informatyki

Praca dyplomowa inżynierska

na kierunku Informatyka
w specjalności Sztuczna Inteligencja

Zastosowanie metod widzenia komputerowego do automatycznej oceny
prawdomówności

Filip Ryniewicz

Numer albumu 325215

promotor
dr inż. Krystian Radlak

WARSZAWA 2026

Zastosowanie metod widzenia komputerowego do automatycznej oceny prawdomówności

Streszczenie

Celem pracy było zbadanie skuteczności metod wizji komputerowej w automatycznej ocenie prawdomówności osób na podstawie analizy nagrań wideo. W pracy przedstawiono różne podejścia do problemu oraz utworzono docelowe narzędzie umożliwiające inferencję modelu uczenia maszynowego na nagraniach zamieszczonych przez użytkownika końcowego w celu oceny prawdomówności.

W badaniach wykorzystano dwa zbiory danych: Silesian Deception Detection Dataset oraz Real-Life Deception Detection Dataset. Do ekstrakcji cech z nagrań wideo, takich jak punkty charakterystyczne twarzy, przepływ optyczny, prawdopodobieństwa wystąpienia emocji, zastosowano narzędzia z bibliotek: MediaPipe, OpenCV, FacialEmotionRecognition.

Rozważono trzy podejścia modelowe: klasyczny algorytm uczenia maszynowego Random Forest, trójwymiarową sieć neuronową konwolucyjną 3D-CNN oraz finalnie wybraną dwukierunkową sieć rekurencyjną GRU z mechanizmem uwagi. W procesie uczenia wykorzystano ekstensywną inżynierię cech oraz technikę transfer learningu (uczenie na jednym zbiorze i dotrenowanie na drugim) w celu poprawy zdolności generalizacji modelu.

Przeprowadzone eksperymenty wykazały, że model Random Forest pozwolił na identyfikację kluczowych cech behawioralnych, towarzyszących kłamstwu (osiągając AUC 0.64 na zbiorze laboratoryjnym), natomiast sieć rekurencyjna wykazała swój potencjał po zastosowaniu transferu wiedzy, uzyskując AUC na poziomie 0.76, co znacznie przewyższa zdolności przeciętnego człowieka.

Integralną częścią pracy jest aplikacja webowa oparta na frameworku Streamlit, pozwalająca użytkownikowi na wgranie pliku wideo i otrzymanie predykcji modelu.

Wyniki badań sugerują, że metody wizji komputerowej mogą wspomagać proces oceny wiarygodności wypowiedzi, jednak potencjalne praktyczne zastosowania wymagają bardzo dużo reprezentatywnych danych treningowych wysokiej jakości.

Słowa kluczowe: wizja komputerowa, sieci neuronowe, przetwarzanie video, ocena prawdomówności, trening modeli, sztuczna inteligencja

Application of computer vision methods for automatic assessment of truthfulness

Abstract

The aim of the thesis was to examine the effectiveness of computer vision methods in the automatic assessment of veracity based on video recording analysis. The thesis presents various approaches to the problem and describes the development of a tool enabling machine learning model inference on recordings provided by the end user.

The research utilized two datasets: the Silesian Deception Detection Dataset and the Real-Life Deception Detection Dataset. Tools from libraries such as MediaPipe, OpenCV, and FacialEmotionRecognition were used for feature extraction from video recordings, including facial landmarks, optical flow, and emotion probabilities.

Three modeling approaches were considered: the classic Random Forest machine learning algorithm, a 3D Convolutional Neural Network (3D-CNN), and the ultimately selected bidirectional recurrent GRU network with an attention mechanism. Extensive feature engineering and transfer learning techniques (pre-training on one dataset and fine-tuning on another) were employed in the training process to improve the model's generalization capabilities.

The conducted experiments showed that the Random Forest model allowed for the identification of the most prominent behavioral features accompanying deception (achieving AUC of 0.64 on the laboratory dataset), while the recurrent network demonstrated its potential through transfer learning, reaching an AUC of 0.76 on the Real-Life Deception Detection Dataset, significantly exceeding the detection capabilities of an average human.

An integral part of the thesis is a web application developed in the Streamlit framework, allowing the user to upload a video file and receive the model's prediction. The results suggest that computer vision methods can support the credibility assessment process; however, potential practical applications require high-quality, representative training data.

Keywords: computer vision, neural networks, video processing, truthfulness assessment, model training, artificial intelligence, deep learning

Spis treści

1 Wstęp	1
1.1 Tło problemu	1
1.2 Przewaga wizji komputerowej nad metodami tradycyjnymi	1
1.3 Cel pracy	2
1.4 Zakres pracy	2
1.5 Układ pracy	3
2 Teoria i przegląd literatury	4
2.1 Psychologiczne aspekty kłamstwa	4
2.1.1 Formalna definicja kłamstwa	4
2.1.2 Teoria obciążenia poznawczego (ang. <i>Cognitive Load Theory</i>)	4
2.1.3 Hipoteza przecieku prawdziwych emocji (ang. <i>Leakage Hypothesis</i>)	5
2.2 Metody wizji komputerowej w analizie twarzy	5
2.2.1 Detekcja obiektów i twarzy	5
2.2.2 Ekstrakcja punktów charakterystycznych twarzy	6
2.2.3 Analiza przepływu optycznego (<i>Optical Flow</i>)	8
2.3 Algorytmy uczenia maszynowego stosowane w rozpoznawaniu kłamstwa	10
2.3.1 Klasyczne metody uczenia maszynowego - Random Forest	10
2.3.2 Rekurencyjne sieci neuronowe (<i>RNN - Recurrent Neural Networks</i>)	11
2.3.3 Alternatywne podejście: trójwymiarowe konwolucyjne sieci neuronowe (3D-CNN)	14
2.4 Przegląd istniejących rozwiązań w detekcji kłamstwa	14
3 Przygotowanie danych i inżynieria cech	15
3.1 Charakterystyka zbiorów danych	15
3.1.1 Silesian Deception Dataset	15
3.1.2 Real-Life Deception Dataset	16
3.2 Potok przetwarzania obrazu	17
3.2.1 Cel potoku i zastosowanie downsamplingu	17
3.2.2 Segmentacja nagrań wideo	17
3.2.3 Lokalizacja i izolacja twarzy	17
3.2.4 Ekstrakcja punktów charakterystycznych i normalizacja geometryczna twarzy	18
3.2.5 Detekcja emocji	18
3.2.6 Wynik potoku przetwarzania	18
3.3 Inżynieria cech	19
3.3.1 Wygładzenie sygnału (Noise reduction and smoothing)	20
3.3.2 Redukcja wymiarowości	20
3.3.3 Analiza dynamiki i prędkości cech (<i>Velocity Features</i>)	20
3.3.4 Augmentacja danych	20
3.4 Strategia podziału danych i walidacji	21
3.5 Normalizacja i przygotowanie sekwencji	21
3.5.1 Normalizacja danych (<i>MinMaxScaling</i>)	21

3.5.2 Przygotowanie sekwencji	22
4 Implementacja i architektura modeli klasyfikacyjnych	23
4.1 Środowisko programistyczne i wykorzystane biblioteki	23
4.2 Model bazowy - Random Forest	24
4.2.1 Przygotowanie danych i wektor wejściowy	24
4.2.2 Konfiguracja modelu	24
4.3 Proponowana architektura głęboka	24
4.3.1 Konfiguracja warstwy rekurencyjnej BiGRU	25
4.3.2 Mechanizm uwagi	25
4.3.3 Moduł klasyfikacyjny i wyjście modelu	26
4.4 Procedura treningowa i optymalizacja	26
4.4.1 Funkcja straty i optymalizator	26
4.4.2 Polityka Learning Rate (OneCycle)	27
4.4.3 Inicjalizacja biasu	27
4.4.4 Walidacja i wybór metryki	27
4.4.5 Kontrola pętli treningowej	27
4.4.6 Podsumowanie hiperparametrów	28
5 Przeprowadzone eksperymenty i analiza ich wyników	29
5.1 Spis przeprowadzonych eksperymentów	29
5.2 Analiza skuteczności modelu bazowego i ważności cech	29
5.2.1 Analiza ważności cech	30
5.3 Weryfikacja techniczna modelu autorskiego (<i>Overfit Check</i>)	32
5.4 Analiza procesu uczenia i skuteczności modelu autorskiego	33
5.4.1 Dynamika procesu uczenia (<i>Learning Curves</i>)	34
5.4.2 Wyniki na zbiorze testowym i porównanie z modelem bazowym	35
5.4.3 Analiza macierzy pomyłek (<i>Confusion Matrix</i>) i histogramu prawdopodobieństwa	36
5.5 Transfer wiedzy (ang. <i>transfer learning</i>)	37
5.5.1 Ewaluacja zero-shot	37
5.5.2 Procedura transferu wiedzy i analiza jego wyników	38
5.5.3 Analiza katastroficznego zapominania	41
6 Implementacja systemu w postaci aplikacji webowej	42
6.1 Analiza wymagań systemowych	42
6.1.1 Wymagania funkcjonalne	42
6.1.2 Wymagania нефunkcjonalne	42
6.2 Założenia projektowe i wybór technologii	43
6.3 Architektura aplikacji	43
6.4 Weryfikacja działania i testowanie oprogramowania	45
6.4.1 Testy jednostkowe	46
6.4.2 Testy integracyjne	46
6.5 Prezentacja funkcjonalności i interfejsu użytkownika	46
6.5.1 Konfiguracja i panel sterowania	46
6.5.2 Przebieg procesu inferencji	47

6.5.3 Prezentacja wyników	47
7 Podsumowanie	49
7.1 Synteza zrealizowanych zadań	49
7.2 Kluczowe wnioski z badań	49
7.3 Ocena praktycznego zastosowania opracowanego systemu	50
7.4 Proponowane kierunki rozwoju	50
Bibliografia	51
Wykaz symboli i skrótów	54
Spis rysunków	54
Spis tabel	55

1 Wstęp

Żyjemy w społeczeństwie opartym na informacji i zaufaniu. Wiedza w dużej mierze przekazywana jest werbalnie, a prawdomówność rozmówców stanowi fundament efektywnej komunikacji międzyludzkiej. Niestety, kłamstwo jest nieodłącznym elementem ludzkiej natury i może prowadzić do poważnych konsekwencji poczynając od utraty zaufania wśród bliskich osób, uszczerbku emocjonalnego, aż po poważne utraty finansowe w przypadku oszustw czy korupcji, a nawet zagrożenia dla bezpieczeństwa (np. w kontekście terroryzmu).

Kłamstwo towarzyszy nam każdego dnia - zarówno w relacjach prywatnych, biznesowych, na lotniskach i w sądach. Ludzie kłamią z różnych powodów, takich jak unikanie kary, osiąganie korzyści materialnych czy ochrona własnego wizerunku. W związku z licznymi negatywnymi skutkami nieszczerości, rozwój metod umożliwiających nieinwazyjną i obiektywną weryfikację prawdomówności ma istotne znaczenie w wielu dziedzinach, takich jak wymiar sprawiedliwości, bezpieczeństwo narodowe czy psychologia sądowa.

1.1 Tło problemu

Mimo wszechobecności zjawiska kłamstwa, ludzie wykazują zaskakująco niską skuteczność w jego rozpoznawaniu. Badania psychologiczne wskazują, że przeciętny człowiek potrafi poprawnie odróżnić prawdę od kłamstwa zaledwie w około 54% przypadków [1]. Wynik ten jest zbliżony do losowego rzutu monetą, co podkreśla trudność tego zadania nawet dla doświadczonych obserwatorów. Niska skuteczność wynika z wielu czynników, takich jak umiejętność kontrolowania mimiki twarzy i mowy ciała przez osoby kłamiące, a także zjawiska tzw. *truth bias* (tendencji do zakładania, że rozmówca mówi prawdę). Często także, niewerbalne sygnały kłamstwa są subtelne i trudne do wychwycenia gołym okiem. Ograniczenia te stwarzają potrzebę opracowania zautomatyzowanych, obiektywnych narzędzi wspierających proces decyzyjny w ocenie prawdomówności.

Podstawą teoretyczną dla budowy takich systemów jest hipoteza *przecieku informacji* (ang. *leakage hypothesis*), spopularyzowana m.in. przez Paula Ekmana [2]. Zakłada ona, że proces kłamania wiąże się ze zwiększonym obciążeniem poznawczym (ang. *cognitive load*). Mózg osoby kłamiącej musi jednocześnie konstruować fałszywą narrację, hamować prawdziwe informacje oraz kontrolować mimikę i mowę ciała, co prowadzi do nieświadomego „wycieku” prawdziwych emocji w formie mimowolnych, trwających ułamki sekund mikroekspresji twarzy, które są trudne do świadomego ukrycia [3], ale i praktycznie niemożliwe do wychwycenia bez specjalistycznego treningu lub wsparcia technologicznego.

1.2 Przewaga wizji komputerowej nad metodami tradycyjnymi

Tradycyjne metody instrumentalnej detekcji kłamstwa, takie jak poligraf (wariograf), opierają się na pomiarze fizjologicznych reakcji organizmu: tętna, potliwości skóry, ciśnienia krwi czy częstotliwości oddechu. Choć skuteczne, metody te są inwazyjne - wymagają podłączenia aparatury do ciała badanej osoby oraz obecności wykwalifikowanego operatora, co ogranicza ich zastosowanie w praktyce.

W przeciwieństwie do metod tradycyjnych, metody oparte na *wizji komputerowej* (ang. *computer vision*) oferują podejście bezkontaktowe. Wykorzystanie kamer oraz algorytmów uczenia maszynowego pozwala na analizę nagrań wideo osób bez konieczności ingerencji fizycznej. Badaniu podlegają: subtelne zmiany w mimice twarzy, ruchy oczu, gesty rąk czy postawa ciała, które mogą dostarczać wskazówek dotyczących prawdomówności. Takie podejście jest mniej stresujące dla badanych, bardziej dyskretne i może być stosowane w szerszym zakresie sytuacji, np. podczas rozmów kwalifikacyjnych, przesłuchań czy monitoringu bezpieczeństwa.

Gwałtowny rozwój technik głębokiego uczenia (ang. *deep learning*) w ostatniej dekadzie, a w szczególności postępy w dziedzinie detekcji obiektów (np. modele YOLO [4] użyte w ramach tej pracy) oraz analizy sekwencji czasowych (sieci rekurencyjne LSTM [5], GRU), otworzył nowe możliwości w zakresie analizy zachowań niewerbalnych. Współczesne algorytmy potrafią wykrywać i interpretować subtelne wzorce w ogromnych zbiorach danych wideo, wyłapując sygnały zupełnie niedostępne dla ludzkiej percepcji. Niniejsza praca wpisuje się w ten nurt badawczy, eksplorując potencjalne zastosowania wizji komputerowej do automatycznej oceny wiarygodności wypowiedzi na podstawie cech wizualnych.

1.3 Cel pracy

Głównym celem niniejszej pracy inżynierskiej jest zaimplementowanie kompletnego systemu informatycznego, wykorzystującego metody wizji komputerowej i algorytmy sztucznej inteligencji do wspierania człowieka w ocenie wiarygodności wypowiedzi na podstawie analizy nagrań wideo. Cel ten został zrealizowany w aspekcie badawczym oraz aplikacyjnym.

Celem badawczym było zbadanie, czy cechy wizualne (mimika, ruchy głowy i reszty ciała, itd.) są wystarczająco informatywne, aby umożliwić skuteczną klasyfikację z dokładnością przewyższającą ludzką percepcję. W tym celu przeprowadzono eksperymenty porównujące różne podejścia modelowe oraz metody ekstrakcji cech z filmów. Dodatkowo, zbadano wpływ technik takich jak transfer learning czy inżynieria cech na zdolność generalizacji modeli.

Celem inżynierskim było zaprojektowanie i utworzenie aplikacji webowej, umożliwiającej użytkownikowi bez technicznego wykształcenia wgranie nagrania wideo i otrzymanie predykcji modelu dotyczącej prawdomówności wypowiedzi.

1.4 Zakres pracy

Realizacja zamierzonego celu wymagała przeprowadzenia szeregu prac badawczych i programistycznych. Zakres pracy obejmował:

1. analizę literatury dotyczącej psychologii kłamstwa, metod wizji komputerowej oraz algorytmów uczenia maszynowego stosowanych w detekcji kłamstwa.
2. pozyskanie zbiorów danych wideo zawierających nagrania osób mówiących prawdę i kłamiących: [Silesian Deception Detection Dataset \(SDDD\)](#) [6] oraz [Real-Life Deception Detection Dataset \(RLDDD\)](#) [7].
3. utworzenie potoku przetwarzania danych (*pipeline*) przekształcającego surowe nagrania wideo na wektory cech wizualnych przystosowanych do trenowania modeli sztucznej inteligencji i zastosowanie go na wyżej wymienionych danych.
4. budowę i trening modeli, m.in.:

- implementacja wybranych architektur modeli
 - zaprojektowanie efektywnego procesu uczenia, poprzez dobór odpowiednich funkcji straty, optymalizatorów i strategii treningowych.
5. przeprowadzenie eksperymentów badawczych, obejmujących:
- strojenie hiperparametrów
 - analizę ważności cech
 - inżynierię cech
 - weryfikację skuteczności techniki *transfer learningu*
 - porównanie skuteczności różnych rozwiązań
6. stworzenie interfejsu dla użytkownika końcowego z wykorzystaniem frameworku Streamlit, umożliwiającego praktyczne zastosowanie finalnie wybranego rozwiązania.

1.5 Układ pracy

Niniejsza praca została podzielona na **siedem rozdziałów**, których treść układa się w logiczny ciąg wprowadzający czytelnika w tematykę, poczynając od podstaw wymaganych do zrozumienia dalej opisanych zagadnień, a kończąc na szczegółowym omówieniu przeprowadzonych badań i uzyskanych wyników.

- **Rozdział drugi** zawiera podstawy teoretyczne dotyczące psychologii kłamstwa, metod wizji komputerowej, zagadnień związanych z uczeniem maszynowym, a także przegląd istniejących rozwiązań w dziedzinie automatycznej detekcji kłamstwa (*state of the art*).
- **Rozdział trzeci** skupia się na danych wykorzystywanych w pracy, opisując zbiory danych, techniki ekstrakcji cech wizualnych oraz proces przystosowania danych do trenowania modeli, poczynając od segmentacji wideo na próbki treningowe, poprzez cały potok przetwarzania, aż po ich normalizację.
- W **rozdziale czwartym** przedstawione są architektury i szczegóły implementacyjne modeli wykorzystanych do klasyfikacji binarnej nagrań, opisany jest proces ich trenowania oraz zastosowane techniki poprawiające zdolność generalizacji modeli.
- **Rozdział piąty** składa się z opisu przeprowadzonych eksperymentów badawczych, analizy uzyskanych wyników, w tym: metryk skuteczności, analizy ważności cech, wpływu inżynierii cech i transfer learningu na efektywność modeli.
- **Rozdział szósty** poświęcony jest praktycznemu zastosowaniu opracowanego rozwiązania, opisując implementację i funkcjonalności aplikacji webowej, utworzonej w celu przedstawienia możliwości systemu użytkownikowi nieposiadającemu wiedzy technicznej.
- W **rozdziale siódmym** zawarte są wnioski końcowe z realizacji projektu, ocena osiągniętych rezultatów oraz propozycje dla dalszego rozwoju i badań w tym obszarze.

2 Teoria i przegląd literatury

Niniejszy rozdział ma na celu zdefiniowanie podstaw teoretycznych, które zostały bezpośrednio wykorzystane w dalszych rozdziałach pracy i ma służyć jako wprowadzenie do zagadnień związanych z tematyką kłamstwa i jego automatycznej detekcji z użyciem wizji komputerowej oraz uczenia maszynowego. Ten projekt naturalnie łączy wiedzę z różnych dziedzin naukowych, w tym psychologii behawioralnej, inżynierii oprogramowania i sztucznej inteligencji. Dlatego, aby zapewnić czytelnikowi pełne zrozumienie kontekstu i podstaw teoretycznych, rozdział ten obejmuje zagadnienia ze wszystkich tych obszarów.

Na początku omówione zostanie w jaki sposób kłamstwo formułowane jest w ludzkim umyśle i po czym można rozpoznać jego zaistnienie w zachowaniu niewerbalnym. Następnie, wytłumaczone zostaną podstawowe metody wizji komputerowej wykorzystywane do analizy cech, takich jak mimika twarzy czy ruchy ciała. Kolejna sekcja przedstawi działanie wybranych algorytmów uczenia maszynowego, które zostały zastosowane w pracy jako modele klasyfikacyjne. Na koniec zaprezentowany zostanie przegląd istniejących już rozwiązań w dziedzinie automatycznej analizy wiarygodności wypowiedzi (*state of the art*), co stanowi punkt odniesienia dla podejścia zaproponowanego w tej pracy.

2.1 Psychologiczne aspekty kłamstwa

2.1.1 Formalna definicja kłamstwa

Według Alderta Vrija kłamstwo definiuje się jako „świadomą próbę wywołania u innej osoby przekonania, które sam mówiący uważa za nieprawdziwe” [8]. Oznacza to, że akt skłamania wiąże się z intencją - zwykła pomyłka nie jest kłamstwem w sensie psychologicznym, pomimo iż wypowiedzane słowa są fałszywe. Podobnie, aktor grający pewną rolę także mówi nieprawdę, ale nie uznaje się tego za kłamstwo, gdyż widzowie na wstępie zakładają, że sztuka niekoniecznie jest faktem. Psychologicznie takie sytuacje są zupełnie odmienne dla mózgu człowieka wypowiadającego się, co przejawia się w jego zachowaniu. Różnice te mogą być wychwycone i sklasyfikowane przez system wizyjny nieznający kontekstu wypowiedzi ani obiektywnej prawdy.

2.1.2 Teoria obciążenia poznawczego (ang. *Cognitive Load Theory*)

Miron Zuckerman zaproponował teorię obciążenia poznawczego zakładającą, że kłamstwo jest znacznie bardziej wymagające mentalnie niż mówienie prawdy [9]. Kłamca musi jednocześnie tworzyć spójną i wiarygodną fałszywą narrację, zapamiętując jej szczegóły, tym samym hamując prawdę. Dodatkowo, stara się kontrolować swoją mowę ciała, żeby zminimalizować ryzyko ujawnienia nieszczerości, obserwując w tym samym czasie reakcję rozmówcy.

Próba równoczesnego skupienia się na tylu zadaniach prowadzi do przeciążenia mózgu, który w efekcie ogranicza zasoby przeznaczone na nieświadomą kontrolę ciała. Skutkuje to widocznymi zmianami mowy ciała, m.in. w częstotliwości mrugania, które staje się rzadsze w czasie wypowiedzania kłamstwa (efekt skupienia), a chwilę po skończeniu wypowiedzi następuje znaczne jego przyspieszenie (efekt kompensacji). Ponadto, ciało oraz głowa kłamiącego wykazują się nienaturalną sztywnością - zastygają one w bezruchu na rzecz podświadomej próby uniknięcia wykonania gestu, który mógłby zasugerować rozmówcy, iż jest on okłamywany.

2.1.3 Hipoteza przecieku prawdziwych emocji (ang. *Leakage Hypothesis*)

Kłamstwo naturalnie wiąże się z intensywnymi emocjami, takimi jak stres czy strach (przed ujawnieniem nieszczerości), ale także ekscytacja i satysfakcja (z udanego wprowadzenia rozmówcy w błąd, tzw. *duping delight* [3]). Hipoteza przecieku sformułowana przez Paula Ekmana i Wallace’a Friesena mówi, że prawdziwe emocje „wyciekają” nawet przy próbie świadomego ich ukrycia [2]. Wypowiadane słowa są łatwo kontrolowane, jednak mimika twarzy, ton głosu i ruchy ciała mogą odzwierciedlać stan emocjonalny bez wiedzy ani kontroli człowieka.

Ruchy twarzy mają dwoistą naturę. Z jednej strony, mięśnie są sterowane świadomie, dzięki czemu możemy się uśmiechnąć do zdjęcia lub zaśmiać się na zawołanie. Z drugiej strony mięśnie te mogą być też sterowane podświadomie, czego przykładem jest prawdziwy uśmiech angażujący mięśnie z okolic oczu. Kłamstwo powoduje konflikt między tymi systemami, co skutkuje wystąpieniem zjawiska bardzo szybkich nieświadomych ruchów mięśni twarzy, zwanych **mikroekspresjami**. Prawdziwa emocja pojawia się na ułamek sekundy (od 1/25 do 1/5 sekundy), zanim człowiek zdąży przybrać „maskę”, za którą ją schowa. W badaniach łączących nieszczerłość z ekspresjami niewerbalnymi twarzy, aż 98,3% uczestników wykazało „przeciek”, próbując ukryć intensywną emocję [10]. Ponadto, dowiedziono, że mechanizm ten jest uniwersalny kulturowo [11] i biologicznie [12] (co wykazano m.in. w eksperymentach z udziałem odizolowanych społeczności w Papui-Nowej Gwinei), przez co może być zaobserwowany u ludzi z różnych krajów i kultur.

Z powodu niezwykle krótkiego okresu trwania tego zjawiska jest ono często niewykrywalne dla ludzkiego oka. Natomiast kamera nagrywająca chociażby w 30 klatkach na sekundę wyłapie ten jeden moment, co pozwala na analizę poklatkową - zatrzymanie czasu i zauważenie tego, co dla oka jest zaledwie mignięciem.

2.2 Metody wizji komputerowej w analizie twarzy

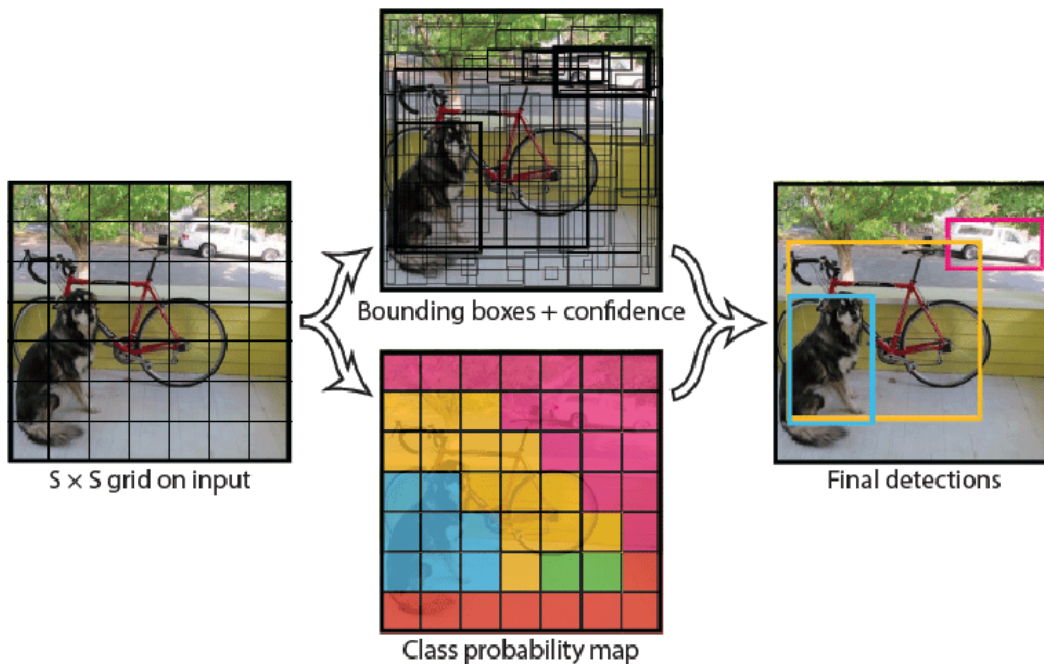
2.2.1 Detekcja obiektów i twarzy

Na początek, ważne jest sprecyzowanie różnicy między klasyfikacją a detekcją obiektów. Klasyfikacja odpowiada na pytanie: „Czy dany obiekt (twarz) znajduje się na obrazie?”. Detekcja natomiast nie tylko klasyfikuje zbiór pikseli jako dany obiekt, ale i tworzy ramkę ograniczającą go o współrzędnych $[x, y, w, h]$, gdzie x i y to współrzędne lewego górnego wierzchołka ramki, a w i h to odpowiednio jej szerokość i wysokość. Informacje uzyskane z użycia detekcji obiektów mogą zostać użyte do redukcji rozmiaru klatki z nagrania do minimalnego regionu zawierającego twarz - obszar zainteresowania (ang. *region of interest*), dzięki czemu znaczna ilość szumu (tło) zostaje pominięta.

Klasyczne metody detekcji twarzy na obrazie, takie jak kaskady Haara były rewolucyjne 20 lat temu - w momencie ich utworzenia przez Paula Violę i Michaela J Jonesa [13]. Są one szybkie, lecz mało dokładne w przypadku obrotów głowy i słabego oświetlenia obiektu. Współczesną alternatywą są metody oparte na konwolucyjnych sieciach neuronowych. Są one odporne na obroty, niekorzystne oświetlenie, a nawet przesłonięcia obiektu.

Najbardziej obiecującym rozwiązaniem jest architektura YOLO („*You Only Look Once*”) [4], którego zaletą jest zaskakująca szybkość wykrywania obiektów. Modele te „patrzą” na

obraz tylko jeden raz, co wprowadza ich przewagę nad starszymi modelami (np. R-CNN), które działały dwuetapowo. Najpierw znajdowały one na obrazie regiony, w których mógł występować wykrywany obiekt, a następnie sprawdzały każdy z tych regionów. Niosło to za sobą niemały narzut wydajnościowy. YOLO łączy te dwa etapy w jeden, przetwarzający cały obraz naraz. Jak przedstawiono na schemacie [Rysunek 1](#), sieć dzieli obraz na siatkę $S \times S$ komórek, z których każda jest odpowiedzialna za wykrycie obiektu, jeśli jego środek znajduje się w jej wnętrzu.



Rysunek 1. Schemat architektury sieci YOLO. Źródło: [14]

Dzięki temu, YOLO jest ekstremalnie szybkie i znajduje zastosowanie nawet w przetwarzaniu wideo w czasie rzeczywistym.

W pracy wykorzystano model YOLO w wersji ósmej implementacji Ultralytics [15], które jest obecnie standardem przemysłowym. Wprowadza ono wiele usprawnień nad wersją przedstawioną w oryginalnych publikacjach, m.in. uproszczoną architekturę i poprawioną zdolność generalizacji na obiekty o nietypowych kształtach (jak twarze). Model ten służy do precyzyjnego zlokalizowania twarzy na klatce, co pozwala na jej wykadrowanie przed przekazaniem do kolejnych etapów potoku przetwarzania.

2.2.2 Ekstrakcja punktów charakterystycznych twarzy

Sam obraz twarzy to tylko zbiór pikseli. Komputer interpretuje mimikę twarzy jako zmianę jasności składowych koloru pikseli, a nie kształty. Biorąc pod uwagę fakt, że pojedyncza klatka nagrania o przeciętnej rozdzielczości to ponad milion pikseli, analiza surowych danych jest nieefektywna.

Ekstrakcja punktów charakterystycznych twarzy (ang. *landmark extraction*) pozwala na znaczną redukcję wymiarowości bez znaczącej utraty informacji. Z pojedynczej klatki wyciągane

są współrzędne 478 kluczowych punktów morfologicznych (np. kąciaki oczu, obrys ust, czubek nosa i łuki brwiowe).

W tym celu, w pracy została zastosowana biblioteka MediaPipe utworzona przez Google [16]. Jest to wieloplatformowy framework do budowania potoków przetwarzania multimediów. Mechanizm jego działania oparty jest na dwuetapowym procesie:

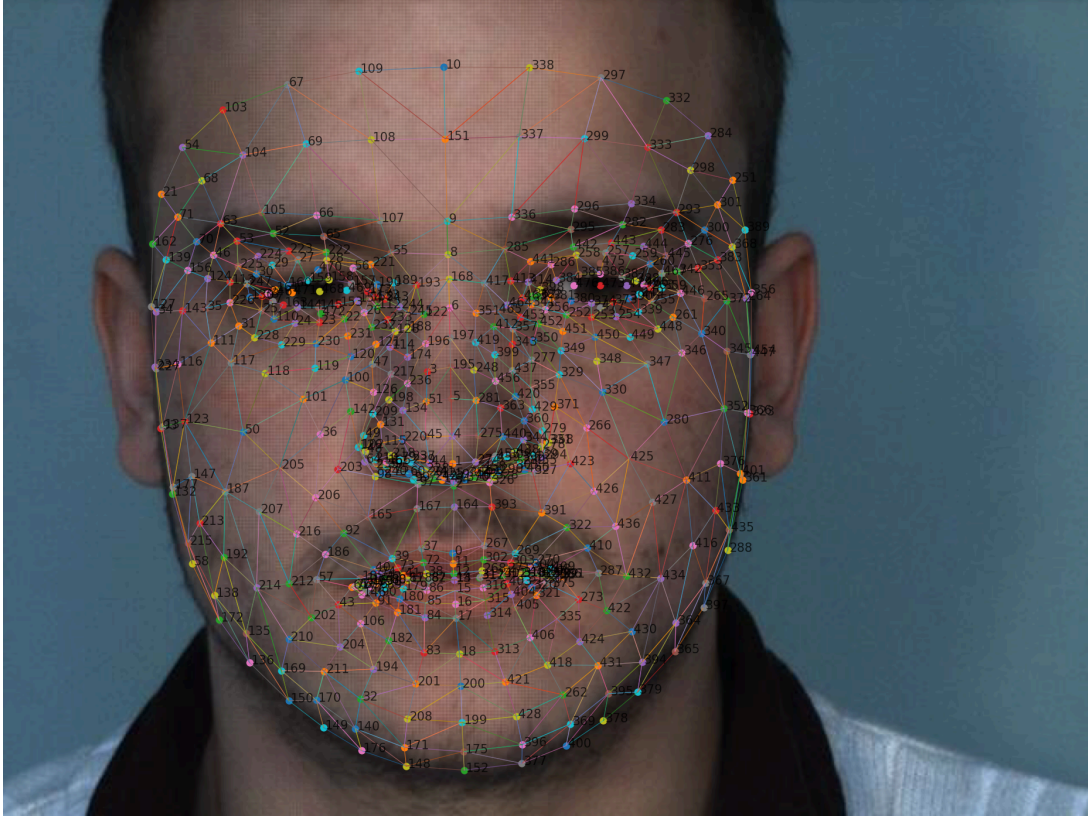
1. Na pierwszej klatce z użyciem lekkiego modelu (BlazeFace) wykrywana jest twarz i wycinany jest prostokąt ograniczający ją.
2. Na kolejnych klatkach model landmarków działa już na wyciętym prostokącie i ponawia detekcję tylko, jeśli twarz zostanie zgubiona, np. przy gwałtownym ruchu głowy.

Dzięki takiemu podejściu, rozwiązanie to jest wysoce optymalne obliczeniowo i działa w czasie rzeczywistym.

Model Face Mesh [17] zbudowany w oparciu o architekturę ResNet tworzy siatkę 3D, składającą się nie tylko ze współrzędnych x i y punktów charakterystycznych, ale także głębokości względem środka głowy z . To pozwala na oszacowanie czy twarz jest przechylona względem kamery. Siatka składa się z 478 precyzyjnie zlokalizowanych punktów. Jej gęstość wyróżnia ją na tle alternatywnych rozwiązań, które generują znacznie mniej punktów (68 w przypadku OpenFace), co pozwala na dokładniejszą analizę, kluczową dla wykrywania mikroekspresji i „przecieku” emocji zgodnie z teorią Ekmana. Dodatkowo wykorzystany jest mechanizm uwagi [18], który pozwala modelowi na skupienie się na kluczowych, „trudnych” obszarach (oczy i usta), co zapewnia wysoką precyzję.

Na podstawie siatki punktów wygenerowanej przez MediaPipe wyliczane są geometryczne cechy pochodne w celu:

- wykrywania mrugnięć (wskaźnik EAR - *Eye Aspect Ratio*), co umożliwia zastosowanie wskaźników opartych na teorii obciążenia poznawczego
- wykrywania otwarć ust (wskaźnik MAR - *Mouth Aspect Ratio*), pozwalając stwierdzić kiedy wypowiedane są słowa
- wyliczania kątów Eulera (*Head Pose Estimation*), opisujących rotację głowy w trzech osiach.



Rysunek 2. Siatka punktów wygenerowana przez MediaPipe. Źródło: opracowanie własne korzystając z nagrania z SDDD

2.2.3 Analiza przepływu optycznego (*Optical Flow*)

Choć wielce informatywne, położenie punktów charakterystycznych twarzy nie jest wystarczające do wartościowej analizy. Charakteryzują one kształt i ułożenie kluczowych elementów mimicznych, ale zupełnie pomijają teksturę i dynamikę ruchu.

Przepływ optyczny definiowany jest jako wzorec pozornego ruchu obiektów, powierzchni i krawędzi w scenie wizualnej, wynikający z relatywnego ruchu między obserwatorem (kamerą) a sceną [19]. W kontekście wizji komputerowej, obraz traktowany jest jako dwuwymiarowe pole wektorowe, gdzie każdemu pikselowi (x, y) przypisywany jest wektor prędkości (u, v) opisujący jego przemieszczenie $\Delta x, \Delta y$ w czasie. Pozwala to na określenie w jakim kierunku i z jaką szybkością porusza się dany punkt.

Algorytmy obliczające przepływ optyczny opierają się na założeniu o stałej jasności - piksel przemieszczający się z punktu A do punktu B zachowuje swój kolor i jasność.

Wzór opisujący tę zależność przyjmuje postać:

$$I(x, y, t) \approx I(x + \Delta x, y + \Delta y, t + \Delta t) \quad (1)$$

W rzeczywistych nagraniach powszechnie występuje zmienne oświetlenie, co oczywiście wpływa na intensywność pikseli. Z tego powodu, przed obliczeniem przepływu optycznego stosuje się konwersję klatek do skali szarości.

Metody obliczania tego wskaźnika dzieli się na dwie kategorie:

- **Metody rzadkie** (*sparse*), które śledzą tylko wybrane, obiecujące punkty (krawędzie, narożniki). Są one szybkie, ale z powodu pominięcia „płaskich obszarów”, gubią informacje o subtelnych ruchach na gładkich powierzchniach (jakimi są np. policzki lub czoło). Wystąpienie mikroekspresji może zostać pominięte, co eliminuje sens użycia ich w detekcji kłamstwa.
- **Metody gęste** (*dense*), obliczające wektor prędkości dla każdego piksela obrazu. Takie podejście jest zdecydowanie bardziej wymagające obliczeniowo, ale pozwala wykryć ruch nawet tam, gdzie nie występują wyraźne punkty charakterystyczne, dzięki czemu ma potencjał na wykrycie wystąpienia mikroekspresji.

Ze względu na wyżej wymienione ograniczenia metod rzadkich, w niniejszej pracy zdecydowano się na użycie implementacji algorytmu Gunnara Farnebacka dostępnej w bibliotece OpenCV [20]. Jest to metoda gęsta pozwalająca na wylapanie każdego niuansu. Algorytm ten aproksymuje sąsiedztwo każdego piksela korzystając z wielomianów kwadratowych i analizuje ich ruchy w kolejnych klatkach.

Analiza przepływu optycznego jest niezwykle przydatna w detekcji kłamstwa i pozwala na dostrzeżenie tego, do czego nie sprawdza się analiza punktów charakterystycznych. Niska jego wartość może wskazywać na „zastyganie” opisane wcześniej w sekcji dotyczącej teorii obciążenia poznawczego. Nagły skok w jego magnitudzie natomiast, może sugerować mikroekspresję [21], które są zbyt subtelne do wykrycia przez FaceMesh.



Rysunek 3. Wizualizacja gęstego przepływu optycznego. Po lewej: oryginalna klatka wideo (wykadrowana twarz). Po prawej: wizualizacja wektorów ruchu. Czarne tło oznacza brak istotnego ruchu (efekt zastosowanego progowania), natomiast barwne plamy wskazują na dynamikę w rejonie oczu, nosa i ust. Źródło: opracowanie własne z użyciem nagrania z SDDD.

2.3 Algorytmy uczenia maszynowego stosowane w rozpoznawaniu kłamstwa

2.3.1 Klasyczne metody uczenia maszynowego - Random Forest

Do zrozumienia działania algorytmu Random Forest (Las Losowy), kluczowe jest poznanie idei drzew decyzyjnych [22]. Jest to hierarchiczna struktura danych, służąca do podejmowania decyzji poprzez zadawanie sekwencji pytań o wartość cech (np. „Czy wartość wskaźnika EAR jest większa od 0,2?”). Powszechnie wizualizowane jest jako odwrócone drzewo, z korzeniem na górze, a liśćmi na dole, gdzie węzłami są kolejne pytania tworzące podział, a liście to finalne predykcje.

Algorytm ten dzieli przestrzeń cech za pomocą prostych reguł decyzyjnych. Podział następuje w sposób rekurencyjny - w każdym węźle wybierany jest podział maksymalnie separujący klasy, maksymalizując zysk informacyjny (ang. *information gain*) lub minimalizując entropię, opierając się np. na indeksie Gini.

Mechanizm działania drzewa decyzyjnego jest bardzo intuicyjny i łatwy do interpretacji. Jednak pojedyncze drzewa decyzyjne mają tendencję do przeuczenia (*overfitting*) - tworzą skomplikowane struktury, idealnie dopasowując się do danych treningowych, przez co słabo radzą sobie z nowymi danymi.

Choć Random Forest używany jest jako pojedynczy klasyfikator, w istocie stanowi on przykład uczenia zespołowego (*ensemble learning*). Nie jest on modelem monolitycznym, a zbiorem wielu drzew decyzyjnych [23]. Jak wspomniano wcześniej, pojedyncze drzewo obarczone jest ryzykiem występowania błędów. Z tego powodu, tworzy się lasy dające uśredniony wynik dziesiątek lub nawet setek estymatorów, zapewniając większą stabilność i dokładność predykcji [22]. W problemach klasyfikacji, jakim jest właśnie zadanie rozpoznawania kłamstwa, ostateczna decyzja podejmowana jest na podstawie tego, którą klasę wybrała większość drzew.

Kluczowym elementem algorytmu jest metoda **Bootstrap Aggregating** (zwana *Baggingiem*). Każde z drzew w lesie uczy się na nieco odmiennym podzbiorze zbioru treningowego [23], a podział ten dokonywany jest na podstawie losowania ze zwracaniem. Dodatkowo, przy tworzeniu podziału w każdym węźle drzewa, algorytm analizuje tylko losowy podzbiór cech, a nie wszystkie dostępne atrybuty. Mechanizmy te zabezpieczają przed przeuceniem występującym w rozwiązaniach korzystających z jednego drzewa, a także wspierają zdolność do generalizacji na nowych danych.

Random Forest wymaga na wejściu wektora cech o długości niezmienniej wśród wszystkich próbek, co stawia wyzwanie dla analizy nagrań, gdyż znacznie różnią się one długością, w zależności od długości wypowiedzi, tempa mowy i wielu innych czynników. Z tego powodu, przed podaniem próbki danych do modelu wymagane jest ujednolicenie ich długości. Jednym z rozwiązań tego problemu jest wykorzystanie **statystyk opisowych z całego nagrania**. Dla każdej cechy obliczana jest jej średnia, odchylenie standardowe, maksimum i minimum, co skutkuje utratą informacji o sekwencyjności zdarzeń. Model ten nie rozumie pojęcia czasu ani kolejności klatek, jednak zachowuje ogólną charakterystykę zachowania. Uczy się on zależności

między wystąpieniem zjawiska kłamstwa i np. liczbą mrugnięć, a nie rozpoznaje zjawisk opisanych wcześniej, takich jak zmiana częstotliwości mrugnięć w trakcie wypowiedzania kłamstwa.

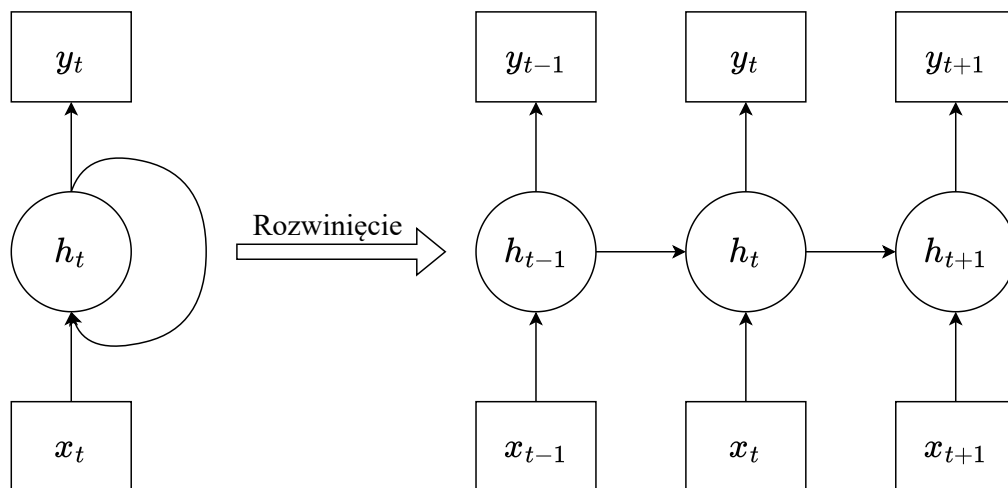
W przeciwieństwie do sieci neuronowych (często określanych jako „czarne skrzynki”, ang. *black box*) opisanych w kolejnych podrozdziałach, Random Forest charakteryzuje się wysoką interpretowalnością. Dzięki analizie ważności cech (*Feature Importance*), z łatwością można precyzyjnie określić, które cechy były najbardziej znaczące przy podejmowaniu decyzji. Pozwala to na empiryczną weryfikację hipotez psychologicznych, dotyczących fizjologicznych wskaźników nieszczerości.

W niniejszej pracy, algorytm ten został użyty jako model bazowy (ang. *baseline*) w celu oszacowania dolnej granicy skuteczności i był traktowany jako punkt odniesienia dla bardziej złożonego, finalnie wybranego modelu.

2.3.2 Rekurencyjne sieci neuronowe (*RNN - Recurrent Neural Networks*)

Kłamstwo to proces dynamiczny naturalnie umiejscowiony w czasie. Ze względu na to, do dokładnego wykrywania go wymagany jest model rozumiejący zależności czasowe i sekwencyjność kolejnych klatek nagrań wideo. Takim modelem jest rekurencyjna sieć neuronowa, przetwarzająca dane jako sekwencje kroków czasowych, gdzie stan w chwili bieżącej jest zależny od stanu go poprzedzającego.

Takie działanie osiągnięte jest za sprawą zastosowanej pętli sprzężenia zwrotnego, w której wyjście neuronu y_t wraca do niego jako wejście w kolejnym kroku czasowym. Stan ukryty (ang. *hidden state*) h_t służy jako „pamięć” sieci. W każdym kroku czasowym t obliczany jest on korzystając z aktualnego wejścia x_t (cechy z bieżącej klatki) i stanu ukrytego z poprzedniego kroku h_{t-1} . W przeciwieństwie do tradycyjnych sieci neuronowych, gdzie każda warstwa ma oddzielne wagi, w RNN te same wagi używane są w każdym kroku czasowym, co pozwala na przetwarzanie sekwencji o dowolnej długości.



Rysunek 4. Schemat sieci rekurencyjnej (RNN). Po lewej: reprezentacja zwinięta z pętlą sprzężenia zwrotnego. Po prawej: reprezentacja rozwinięta w czasie, ukazująca przepływ informacji (stanu ukrytego) przez kolejne kroki sekwencji. Źródło: opracowanie własne na podstawie [24].

Trening takiej sieci bazuje na „rozwinieciu” jej w czasie, tworząc bardzo głęboką sieć, gdzie każda warstwa to jeden krok czasowy. Aby umożliwić naukę sieci, po przepuszczeniu przez nią sekwencji obliczany jest błąd, według którego aktualizowane są wagi, cofając się do początku sekwencji. Jest to algorytm BPTT (*Back Propagation Through Time*). Ponieważ wagi sieci neuronowej często są małymi liczbami (mniejszymi niż 1), mnożenie ich przez siebie setki razy (w przypadku długich sekwencji) w trakcie propagacji wstecznej powoduje wykładnicze malenie gradientu. Nazywane jest to problemem zanikającego gradientu [25], który skutkuje tym, że wagi na początku sekwencji przestają się aktualizować, a w konsekwencji na wynik predykcji długiej sekwencji najmocniej wpływają kroki czasowe z jej końca, a informacja z jej początku „zanika”.

Jak wcześniej zauważono, klasyczne sieci RNN mają krótką pamięć. Problem ten rozwiązują architektury bramkowe, takie jak LSTM (*Long Short Term Memory*) zaprojektowany przez Hochreitera i Schmidhubera w celu usunięcia zanikających gradientów [26]. Wprowadza on dodatkowy przepływ informacji między kolejnymi krokami czasowymi w postaci stanu komórki (ang. *cell state*) C_t , pozwalający pamiętać informację na długich odcinkach sekwencji. Dzięki mechanizmowi bramkowania (ang. *gating*), LSTM potrafi decydować co zapomnieć, a co zapamiętać za pomocą struktur zwanych bramkami. Występują trzy ich rodzaje:

- **Bramka Zapominania** (*Forget Gate*): na podstawie wejścia x_t oraz stanu ukrytego z poprzedniego kroku h_{t-1} decyduje, co wyrzucić ze stanu komórki.
- **Bramka Wejściowa** (*Input Gate*): Decyduje, jakie nowe informacje zapisać w stanie komórki.
- **Bramka Wyjściowa** (*Output Gate*): Decyduje, co z obecnego stanu komórki zapisać do stanu ukrytego, przekazując do kolejnego kroku jako h_t .

Bramki wprowadzają znaczne usprawnienie działania rekurencyjnych sieci neuronowych, ale tym samym LSTM ma niezwykle dużo trenowalnych parametrów, przez co proces jego uczenia jest utrudniony i wymaga większej ilości danych treningowych. Architektura GRU (*Gated Recurrent Unit*) została opracowana przez Kyunghyuna Cho w celu optymalizacji i uproszczenia LSTM [27]. Łączy ona stan ukryty i stan komórki w jedno, ale korzysta z bramek w celu ochrony przed zanikającymi gradientami. W przeciwieństwie do LSTM, gdzie są trzy bramki, w GRU są tylko dwie:

- **Bramka Aktualizacji** (*Update Gate*) z_t : Jednocześnie pełni rolę bramki zapominania i wejściowej z LSTM. Decyduje, w jakim stosunku brać pod uwagę poprzedni stan ukryty i aktualne wejście.
- **Bramka Resetu** (*Reset Gate*) r_t : Pozwala sieci całkowicie porzucić poprzedni stan ukryty, resetując pamięć krótkotrwałą, jeśli uzna go za nieistotny dla aktualnego stanu.

Standardowe sieci rekurencyjne przetwarzają sekwencje chronologicznie, przez co decyzja sieci w danym momencie zależy tylko od kroków czasowych go poprzedzających. Z uwagi na fakt, że w pracy tej analizowane pod kątem prawdziwości są istniejące już nagrania, a celem nie jest utworzenie systemu czasu rzeczywistego, postawiono wzbogacić architekturę GRU o działanie dwukierunkowe, tworząc architekturę Bi-GRU [28]. Składa się ona z dwóch oddzielnych warstw, z których jedna przetwarza nagranie w przód, a druga w tył. W takim podejściu, finalna reprezentacja pojedynczej klatki jest wynikiem konkatencji wektorów wyjściowych obu warstw.

2.3.3 Alternatywne podejście: trójwymiarowe konwolucyjne sieci neuronowe (3D-CNN)

Jako alternatywa do RNN, rozważono użycie trójwymiarowych konwolucyjnych sieci neuronowych (3D-CNN) [30]. Klasyczne sieci konwolucyjne CNN (używane np. w YOLO) operują bezpośrednio na obrazach. Ich trójwymiarowy odpowiednik rozszerza operację splotu (konwolucji) o trzeci wymiar - czas. Takie podejście eliminuje potrzebę skomplikowanego przetwarzania wstępnego, ponieważ sieć bezpośrednio z nagrań uczy się jednocześnie wyglądu obiektów i ich ruchu. Nie jest to jednak rozwiązanie bez wad. Tak skomplikowany model jest bardzo ciężki obliczeniowo, a efektywne wytrenowanie go wymaga tysięcy, a nawet milionów nagrań. Dziedzina detekcji kłamstwa cierpi na deficyt danych, a publicznie dostępne zbiory liczą zaledwie setki próbek. Z tego powodu, zdecydowano się na porzucenie prób modelowania opartych na tej architekturze.

2.4 Przegląd istniejących rozwiązań w detekcji kłamstwa

Początkowe próby zastosowania algorytmów uczenia maszynowego do automatycznej detekcji kłamstwa opierały się na klasycznych metodach wizji komputerowej. Dane zostawały poddane ręcznej ekstrakcji cech geometrycznych i tekstur, a następnie były używane do trenowania klasyfikatorów - maszyn wektorów nośnych (SVM) lub drzew decyzyjnych. Przykładem takiego podejścia jest praca [31], w której wykorzystano deskryptor LBP-TOP (*Local Binary Patterns from Three Orthogonal Planes*) do analizy czasoprzestrzennych zmian tekstury twarzy, używając klasyfikatora SVM. Rozwiązania te, choć przełomowe na tamten okres (ok. 2010-2015), były bardzo podatne na błędy wynikające z nieoptymalnych warunków oświetlenia i przemieszczeń kamery.

Przełom nastąpił wraz z popularyzacją głębokiego uczenia (ang. *deep learning*). Kluczową pracą z tego okresu jest publikacja Pérez-Rosas et al. [32], w ramach której utworzono zbiór Real-life Trial Dataset, zawierający nagrania z prawdziwych rozpraw sądowych. W swoich eksperymentach, autorzy wykorzystali drzewa decyzyjne oraz lasy losowe do klasyfikacji próbek składających się z cech językowych (wyekstrahowanych z transkrypcji) oraz wizualnych: m.in. trajektorie ruchów głowy i dłoni oraz detekcja uśmiechów. Badania te wykazały, że automatyczne systemy oparte na ekstrakcji cech wizualnych są w stanie osiągnąć skuteczność nawet 75%, znacząco przewyższając ludzką percepcję (wcześniej wspomniane 54%).

Współczesne podejścia ewoluowały w stronę analizy multimodalnej, jednocześnie biorąc pod uwagę nie tylko obraz z kamery, ale także dźwięk (między innymi ton głosu) oraz treść wypowiedzi (korzystając z technik przetwarzania języka naturalnego, ang. *NLP - Natural Language Processing*). Prace takie jak Gogate [33] wykorzystywały potężne architektury łączące sieci konwolucyjne CNN z mechanizmami uwagi, pobijając wcześniejsze próby zautomatyzowania procesu oceny prawdopodobieństwa wypowiedzi, osiągając skuteczność rzędu 80%.

Równocześnie istniejący nurt badawczy wskazuje jednak, że to właśnie kanał wizualny dostarcza najwięcej unikalnych i trudnych do sfalszowania sygnałów (w postaci „przecieków emocjonalnych”), na co wskazuje literatura poświęcona detekcji mikroekspresji [34]. Uzasadnia to skupienie się w niniejszej pracy na analizie wideo z wykorzystaniem metod wizji komputerowej w postaci m. in. Optical Flow oraz sekwencyjności rekurencyjnych sieci neuronowych.

3 Przygotowanie danych i inżynieria cech

Jakość i sposób przygotowania danych są w projektach uczenia maszynowego równie ważne jak architektura wykorzystanych modeli. Nawet najbardziej rozbudowany model niczego się nie nauczy, jeśli do treningu dostanie zaszumione i niereprezentatywne dane. W detekcji kłamstwa, gdzie o etykiecie próbki danych świadczą bardzo subtelne sygnały (takie jak mikroekspresje), adekwatne przygotowanie danych jest kluczowe. W tym rozdziale przedstawiono szczegółowy opis wykorzystanych w pracy zbiorów danych, obejmujący metody ich pozyskania oraz techniki wykorzystane w celu uzyskania jak najlepszej reprezentatywności. Następnie, przedstawiony zostanie kompletny potok przetwarzania danych, poczynając od surowych nagrań wideo, a kończąc na tensorach cech zrozumiałych dla sieci neuronowych. Opisana zostanie także strategia podziału danych na zbiory treningowe, walidacyjne i testowe oraz ich normalizacja.

3.1 Charakterystyka zbiorów danych

W pracy wykorzystano dwa zbiory danych, aby sprawdzić zdolność wybranego rozwiązania do nauki na danych przygotowanych w kontrolowanych warunkach laboratoryjnych, jak i jego generalizację w warunkach rzeczywistych (nagranie zbliżone jakością do tych, które będą zamieszczane przez użytkownika końcowego).

3.1.1 Silesian Deception Dataset

Zbiór **SDDD** został użyty jako zbiór podstawowy, z którego model uczy się wskaźników kłamstwa. Został on opracowany przez naukowców z Politechniki Śląskiej [6]. Zawiera nagrania 101 uczestników (studentów), co zapewnia wystarczająco dużą różnorodność biometryczną twarzy. Próbkę danych zostały zebrane w kontrolowanych warunkach, a w szczególności:

- kontrolowane statyczne oświetlenie,
- jednolite tło,
- kamera ustawiona na wprost.

Taki sposób pozyskania nagrań w pełni eliminuje szum tła, pozwalając modelowi na skupienie się wyłącznie na twarzach uczestników. W procesie tworzenia tego zbioru danych została wykorzystana profesjonalna kamera rejestrująca obraz w rozdzielczości 640x480 pikseli z szybkością 100 klatek na sekundę.

Twórcy **SDDD** trafnie zauważyli, że w celu pozyskania nagrań zawierających naturalne wskaźniki nieszczerości trzeba stworzyć środowisko, w którym kłamstwo uczestników będzie znaczące, a nie tylko z góry narzucone. W tym celu, nagrania zostały przeprowadzone jako badanie rzekomych zdolności telepatycznych pewnej osoby. Przed uczestnikiem stał laptop, na którego ekranie wyświetlała się figura geometryczna oraz polecenie „skłam” lub „powiedz prawdę”. Usadzone na przeciwko niego był rzekomy telepata, który miał czytając im w myślach zdecydować czy na ekranie faktycznie jest dana figura geometryczna czy nie. Obecność kamer została uzasadniona wykluczeniem możliwości niewerbalnego podpowiadania telepacie, a jedna z nich była skierowana właśnie na niego w celu zwiększenia immersji tej narracji. Dzięki tym zabiegom, udało się uchwycić zachowania zbliżone do tych, które występują podczas prawdziwego kłamania - spontaniczne, niekontrolowane reakcje mimiczne, mikroekspresje i napięcie emocjonalne.

Nagranie każdego uczestnika zawiera w sobie dziesięć próbek danych. Proces opisany wcześniej był ustrukturyzowany w taki sposób, że uczestnik wypowiadał się dziesięć razy, gdzie pierwsza, druga i ostatnia wypowiedź miała być szczerą, a pozostałe - kłamstwami. Z tego powodu zbiór jest silnie niezbalansowany - 70% przykładów to kłamstwa. Taka struktura nagrań wymaga ręcznej ich segmentacji, w czym pomagają załączone pliki o formacie .eaf, zawierające m.in. informację o tym w jakich przedziałach czasowych zamyka się pojedyncza wypowiedź. W plikach tych zaznaczone są także odstępstwa od scenariusza - momenty, w których zaszła pomyłka i w miejscu kłamstwa pojawiła się prawda, itp. Takie przykłady musiały zostać odrzucone. Po usunięciu wadliwych próbek, zbiór składa się z 924 przykładów.

3.1.2 Real-Life Deception Dataset

W przeciwieństwie do nagrań ze zbioru SDDD przygotowanych w warunkach laboratoryjnych, RLDDD [7] składa się z klipów pochodzących z prawdziwych procesów sądowych (głównie z USA) - fragmenty zeznań świadków oraz wyjaśnień oskarżonych. Zbiór zawiera zaledwie 121 nagrań, lecz ze względu na liczebność klas jest niemal idealnie zbalansowany - 61 kłamstw i 60 wypowiedzi szczerych.

Ze względu na okoliczności powstania nagrań (środowisko sali sądowej bez reżysera ani naukowców), są one niezwykle reprezentatywne. Kłamstwa zawarte w filmach są rzeczywistymi i naturalnymi kłamstwami wysokiej wagi (ang. *High-Stakes Lies*). Przedstawieni w nich ludzie kłamią walcząc o swoją przyszłość, reputację, a nawet życie. Takie wypowiedzi wiążą się z ogromnym obciążeniem poznawczym (ang. *cognitive load*) i silnym stresem, co stwarza idealne warunki do wystąpienia silnych i trudnych do ukrycia mikroekspresji oraz mimowolnych ruchów mięśni twarzy, których nie da się zreplikować w kontrolowanych warunkach eksperymentu laboratoryjnego. Dzięki temu zbiór ten pozwala zweryfikować użyteczność modelu w systemach bezpieczeństwa, gdzie celem jest wykrywanie realnych zagrożeń, a nie akademickiego kłamstwa.

W przeciwieństwie do statycznej kamery i oświetlenia w SDDD, w tym zbiorze z powodu rozproszonego sposobu pozyskania nagrań (różne sale sądowe), cechują się one:

- różnymi kątami kamery (niekoniecznie na wprost),
- zmiennym oświetleniem,
- ruchami kamery,
- przysłonięciami twarzy (np. ręką lub mikrofonem).

To właśnie te ograniczenia sprawiają, że zbiór ten jest idealnym zasobem testowym dla odporności modelu na szum i jego zdolności generalizacji.

Etykiety przykładów z tego zbioru zostały przypisane na podstawie ostatecznych werdyktów sądowych oraz zweryfikowanych przez policję faktów. Choć system sądowniczy nie jest perfekcyjnie wiarygodny i nieomylny, jest to najlepsze przybliżenie prawdy dostępne dla danych rzeczywistych.

Porównanie przykładowych klatek z obu zbiorów zostało przedstawione na obrazku poniżej.



Rysunek 6. Porównanie jakości próbek w wykorzystanych zbiorach danych. Po lewej: przykładowa klatka ze zbioru **SDDD** - widoczne idealne oświetlenie, jednolite tło i statyczna pozycja. Po prawej: przykładowa klatka ze zbioru **RLDDD** - niska rozdzielczość, małe zbliżenie i niejednolite tło.

3.2 Potok przetwarzania obrazu

3.2.1 Cel potoku i zastosowanie downsamplingu

Celem przedstawionego potoku jest transformacja surowego nagrania wideo w ustrukturyzowany zbiór wyekstrahowanych cech numerycznych w postaci wektorów, które mogą być podane do sieci neuronowej. Proces odbywa się klatka po klatce, ale z zastosowaniem downsamplingu. Sieci rekurencyjne nie radzą sobie z bardzo długimi sekwencjami. Z tego powodu zdecydowano się na analizę co n -tej klatki w celu redukcji wymiarowości finalnych wektorów. Pozwoliło to także na ujednolicenie szybkości nagrań pochodzących z obu zbiorów. W przypadku **SDDD** zawierającego nagrania w stu klatkach na sekundę, brano co dziesiątą klatkę, a w przypadku **RLDDD**, gdzie nagrania mają 30 fps (*frames per second*) - co trzecią klatkę. Dzięki temu finalne wektory pochodzące z obu zbiorów mają jednolitą liczbę wartości cech na sekundę (10), a zatem percepcja czasu sieci rekurencyjnej będzie zachowana. W przypadku niezastosowania takiego ujednolicenia, ruchy i zdarzenia z obu zbiorów znacznie różniłyby się szybkością, co mogłoby być nieoptymalne dla skuteczności modelu.

3.2.2 Segmentacja nagrań wideo

W przypadku zbioru **SDDD** kluczowe było odpowiednie podzielenie nagrań na próbki, gdyż oryginalne wideo zawierają dziesięć wypowiedzi, co jest jednoznaczne z dziesięcioma próbkami danych. W tym celu utworzono parser plików **.eaf**, w których zawarte były znaczniki czasowe (ang. *timestamps*) pojedynczych wypowiedzi. Nagrania ze zbioru **RLDDD** natomiast, zostały potraktowane jako pojedyncze segmenty.

3.2.3 Lokalizacja i izolacja twarzy

Wykorzystano model YOLOv8-nano do detekcji twarzy i wycięcia obszaru zainteresowania (region twarzy) z całej klatki. Dzięki temu zabiegowi, tło tworzące nieistotny szum zostaje usunięte. Wykadrowany ROI (*Region of Interest*) zostaje przeskalowany do rozdzielczości 224x224 pikseli, która jest standardem dla większości architektur sieci neuronowych (w tym MediaPipe FaceMesh oraz model używany do detekcji emocji).

3.2.4 Ekstrakcja punktów charakterystycznych i normalizacja geometryczna twarzy

Mediapipe FaceMesh został zastosowany w celu ekstrakcji współrzędnych punktów charakterystycznych twarzy. Na ich podstawie dokonywana jest normalizacja geometrii twarzy - wyznaczane są środki oczu oraz kąt nachylenia głowy i względem ich wykrywana jest rotacja obrazu uzyskująca poziomą linię oczu. Jest to krok wymagany przed podaniem do modelu wykrywającego emocje, który został wytrenowany na zdjęciach frontalnych i wymaga wyrównanej twarzy do poprawnej detekcji.

3.2.5 Detekcja emocji

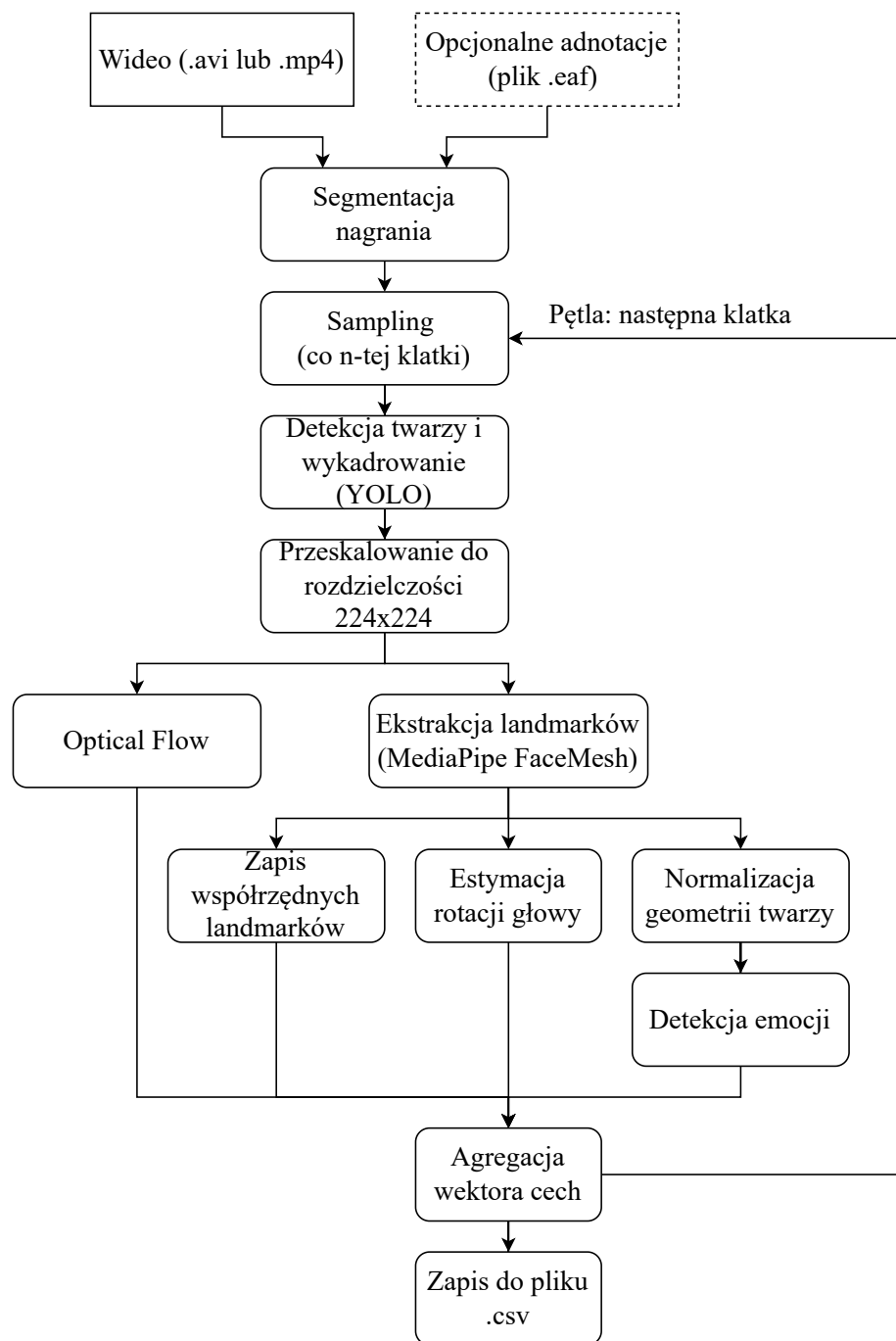
Znormalizowany geometrycznie obraz trafia do modułu rozpoznawania emocji. Do tego zadania została wykorzystana konwolucyjna sieć neuronowa (model Facial Emotion Recognition [35] wytrenowana na zbiorze [36]), która klasyfikuje wyraz twarzy do jednej z siedmiu podstawowych kategorii: złość, obrzydzenie, strach, radość, smutek, zaskoczenie oraz wyraz neutralny. Model zwraca wektor prawdopodobieństw (z dystrybucji Softmax) określający pewność modelu co do wystąpienia każdej z wyżej wymienionych emocji.

3.2.6 Wynik potoku przetwarzania

Z każdej analizowanej klatki uzyskiwany jest wektor cech, w którego skład wchodzi:

- **Globalna pozycja głowy** - współrzędne środka ramki ograniczającej twarz (*bounding box*) oraz jej szerokość. Pozwala to modelowi klasyfikacyjnemu na śledzenie ruchów ciała (np. wiercenie się na krześle lub „zastyganie” wynikające ze stresu związanego z kłamstwem) oraz zmiany dystansu od kamery, które zostałyby utracone w procesie kadrowania samej twarzy.
- **Morfologia twarzy** - współrzędne 478 punktów charakterystycznych z siatki MediaPipe.
- **Pozycja głowy** - estymowane kąty Eulera (Pitch, Yaw, Roll) opisujące rotację głowy w trójwymiarowej przestrzeni.
- **Emocje** - prawdopodobieństwa wystąpienia siedmiu podstawowych emocji.
- **Optical Flow** - średnia i odchylenie standardowe gęstego przepływu optycznego w osiach X i Y obliczone z użyciem algorytmu Farnebacka względem poprzedniej przetworzonej klatki.

Diagram potoku przetwarzania obrazu został przedstawiony na obrazku poniżej.



Rysunek 7. Schemat blokowy potoku przetwarzania obrazu.

3.3 Inżynieria cech

Potok przetwarzania obrazu przedstawiony w poprzedniej sekcji wyciągał informacje z nagrań i przekształcał je na wektory cech. W tym podrozdziale, natomiast, przedstawiony będzie proces transformacji ich w inteligentne wskaźniki, które ułatwią modelom nauczenie się rozpoznawania kłamstwa.

3.3.1 Wygładzenie sygnału (Noise reduction and smoothing)

Surowe dane pochodzące z potoku przetwarzania bywają zaszumione. Klatka po klatce, wartości cech mogą drgać (tzw. *jitter*), a wynika to z ograniczonej dokładności użytych detektorów i algorytmów. Zastosowanie średniej kroczącej (ang. *rolling window average*) o oknie długości 5 klatek na cechach, takich jak pozycja głowy (Pitch, Yaw, Roll) oraz parametry ramki ograniczającej twarz (współrzędne jej środka i jej szerokość) pozwala na usunięcie szumu pomiarowego przy jednoczesnym zachowaniu rzeczywistych ruchów ciała i głowy.

3.3.2 Redukcja wymiarowości

Wektory cech uzyskane z potoku przetwarzania składają się z aż 973 elementów na jedną klatkę z nagrania. Bezpośrednie podanie sekwencji zawierających aż tyle cech, przy tak małej ilości próbek (poniżej 1000) wiąże się z ogromnym ryzykiem przeuczenia - model zapamiętałby dokładnie każdą z próbek ze zbioru treningowego ale zupełnie poległby na nowych danych. Z tego powodu, postanowiono zredukować wymiarowość wektorów. Znaczna większość cech (956) to współrzędne landmarków, więc są one idealną podgrupą cech do zredukowania. Na podstawie współrzędnych punktów charakterystycznych wyliczane są odległości między kluczowymi parami:

- **Oczy** (wskaźnik **EAR** - *Eye Aspect Ratio*): odległość między powiekami,
- **Usta** (**MAR** - *Mouth Aspect Ratio*): ich szerokość i wysokość,
- **Brwi**: odległość brwi od oka.

W ten sposób, zachowując istotną część informacji, udało się zredukować długość wektorów do zaledwie 23 elementów, co stanowi znaczną poprawę względem początkowych 973.

Problemem tego podejścia jest fakt, że te dystanse (w pikselach) zależne są od jakości utworzonego przez YOLO bounding boxa oraz biometrii twarzy danej osoby. Z tego powodu, wszystkie wyliczone dystanse geometryczne dzielone są przez wysokość twarzy zdefiniowaną jako odległość między górnym krańcem czoła a dolnym zwieńczeniem podbródka w danej klatce. Dodatkowo, od każdego wyliczonego dystansu odejmowana jest jego średnia wartość z całego nagrania (*zero-centering*). Dzięki tym zabiegom, model nie będzie uczył się zależności między fizjologią osób a ich szczerością, lecz będzie analizował zmiany tych cech względem stanu spoczynkowego danej osoby, a zatem efektywnie wykrywał mikroekspresje.

3.3.3 Analiza dynamiki i prędkości cech (*Velocity Features*)

Surowe koordynaty obszaru twarzy same w sobie nie są zbytnio informatywne. Fakt, że twarz znajduje się na lewej połowie klatki nie niesie za sobą znaczącej informacji o szczerości wypowiedzi. Obliczenie pierwszej pochodnej (a de facto prędkości zmiany położenia twarzy) pozwala na wykrycie nerwowości ruchów oraz gwałtowności reakcji.

3.3.4 Augmentacja danych

Do finalnego wektora uzyskanego z użyciem wyżej opisanych technik dodawany jest losowy szum gaussowski o odchyleniu standardowym 0,05. Zapobiega to przeuczeniu się sieci na specyficznych, dokładnych wartościach liczbowych i zmusza ją do nauki ogólnych wzorców, co zwiększa odporność na błędy detekcji w warunkach rzeczywistych, a zarazem wspiera regularyzację modelu.

3.4 Strategia podziału danych i walidacji

Dane dzielone są na trzy podzbiory:

- Zbiór treningowy (80% próbek) - jest używany bezpośrednio w trakcie nauki modelu jako jego wejście, służąc do optymalizacji wag modelu,
- Zbiór walidacyjny (10% próbek) - służy do ewaluacji skuteczności modelu w trakcie treningu,
- Zbiór testowy (10% próbek) - używany jest do finalnego sprawdzenia skuteczności wytrenowanego już modelu.

Podział losowy wszystkich próbek skutkowałby prawdopodobnym wystąpieniem nagrań jednej osoby w ponad jednym zbiorze. Takie zjawisko, zwane wyciekami tożsamości/danych (ang. *identity/data leakage*), może mieć katastrofalne konsekwencje - model nauczyłby się mimiki konkretnych osób zamiast uczyć się uniwersalnych wskaźników kłamstwa, co zwiększyłoby jego skuteczność, ale nie w sposób pożądaný - dokładność na nowych danych byłaby wciąż niska. Aby temu zapobiec, dane zostały podzielone z wykorzystaniem podejścia *Subject Independent*, gdzie każdej osobie przydzielony został identyfikator, a następnie nagrania zostały podzielone w rozłączne grupy względem nich.

W przypadku zbioru **SDDD**, ze względu na ustrukturyzowany przebieg pozyskiwania nagrań (każdy uczestnik nagrywał 3 wypowiedzi szczerze i 7 kłamstw), taki podział naturalnie zachowywał globalną dystrybucję etykiet w każdym z podzbiorów. Natomiast dla zbioru **RLDDD**, gdzie każde nagranie przedstawia inną osobę, wymagana była jawna stratyfikacja podziału, aby wymusić równomierny rozkład etykiet w zbiorze treningowym, walidacyjnym i testowym.

3.5 Normalizacja i przygotowanie sekwencji

3.5.1 Normalizacja danych (*MinMaxScaling*)

Różne grupy cech mają bardzo odmienne zakresy wartości. Emocje są prawdopodobieństwami z zakresu $[0, 1]$, a kąty głowy (Pitch, Yaw, Roll) są w stopniach $[-90, 90]$. Podanie takich danych do sieci neuronowej bez skalowania spowodowałoby, że cechy o większych wartościach zdominowałyby proces uczenia, a subtelne sygnały nie byłyby w ogóle brane pod uwagę. W celu wyrównania zakresów wartości cech zastosowano skalowanie liniowe do przedziału $[-1, 1]$ z użyciem `MinMaxScaler` z biblioteki `scikit-learn`. Funkcje aktywacji powszechnie używane w sieciach rekurencyjnych (takie jak tangens hiperboliczny $\tanh$) operują właśnie na takim zakresie. Dopasowanie danych treningowych do tego zakresu przyspiesza trening.

W projekcie zostały użyte 4 skalery do oddzielnego przeskalowania różnych grup cech, odpowiednio kolumny dotyczące:

- Optical Flow,
- emocji,
- rotacji głowy,
- pozycji bounding boxa.

Skalowanie dystansów między punktami charakterystycznymi nie było konieczne, ze względu na opisane wcześniej techniki - wartości już są w odpowiednim zakresie.

Skalery zostały dopasowane (`fit`) wyłącznie na zbiorze treningowym, a następnie przy użyciu wyliczonych już parametrów nastąpiło przekształcenie (`transform`) zbioru testowego i walidacyjnego. Taka strategia zapobiega wyciekowi danych. Gdyby skaler został dopasowany na wszystkich danych, model w trakcie treningu poznałby globalną dystrybucję wartości cech, co sztucznie zawyżyłoby wyniki.

3.5.2 Przygotowanie sekwencji

Nagrania wideo naturalnie mają różne czasy trwania. Sieci neuronowe trenowane paczkami danych (*batch*) wymagają jednak tensorów o regularnych kształtach (prostokątnych macierzy). W celu ujednolicenia długości sekwencji wykorzystano technikę paddingu, w której wszystkie próbki w ramach jednego batcha zostają wydłużone do wyrównania z najdłuższym nagraniem z paczki. Brakujące klatki zostają wypełnione stałą wartością -10 , która została specjalnie wybrana spoza zakresu wartości cech w celu odróżnienia jej od nich. Wartość ta jest traktowana jako pusta informacja. Aby model nie uczył się na sztucznym dopełnieniu, generowany jest dodatkowy wektor (maska) informujący o rzeczywistej długości każdej z sekwencji. Dzięki temu sieć rekurencyjna GRU wie, w którym momencie zatrzymać aktualizację stanu ukrytego dla danej próbki.

Finalnie wektory cech (trzymane wcześniej w numpy arrays) konwertowane są do tensorów PyTorch, które są natywną strukturą danych dla operacji na karcie graficznej GPU, gdzie wykonywany jest trening sieci neuronowych.

4 Implementacja i architektura modeli klasyfikacyjnych

W poprzednim rozdziale szczegółowo opisano dane używane w projekcie oraz sposób ich przetwarzania, poprzez ekstrakcję cech z nagrań wideo, inżynierię cech w celu zwiększenia ich informatywności, kończąc na finalnych tensorach. Celem tego rozdziału jest przedstawienie szczegółów implementacyjnych algorytmów sztucznej inteligencji, które na wejście dostają właśnie te przygotowane dane. Najpierw scharakteryzowane zostanie środowisko programistyczne oraz biblioteki i narzędzia użyte w ramach realizacji projektu. W dalszej kolejności omówiony zostanie model bazowy Random Forest, służący jako punkt odniesienia dla bardziej zaawansowanego rozwiązania. Główna część rozdziału będzie poświęcona szczegółowemu opisowi proponowanej architektury głębokiej sieci neuronowej, opartej na dwukierunkowych jednostkach rekurencyjnych (BiGRU), wzbogaconych o mechanizm uwagi. Rozdział zostanie zakończony analizą procedury treningowej, doboru hiperparametrów oraz technik użytych w celu poprawy zdolności generalizacji modelu.

4.1 Środowisko programistyczne i wykorzystane biblioteki

Całość projektu została zrealizowana w języku programowania Python w wersji 3.10.14. Jest to język dominujący dziedziny Data Science oraz Machine Learning i z tego właśnie powodu zdecydowano się na jego wybór. Specyficzna wersja została dobrana w sposób maksymalizujący liczbę kompatybilnych bibliotek. Jako środowisko pracy wykorzystano Visual Studio Code, a badania były przeprowadzane z użyciem Jupyter Notebooków, pozwalających na interaktywną analizę danych, szybkie i wygodne restartowanie fragmentów kodu oraz wizualizację wyników między komórkami (ang. *inline*). Framework głębokiego uczenia maszynowego PyTorch stanowił główny silnik projektu i został użyty do:

- konstrukcji sieci neuronowych - warstw GRU, Linear, modułu Attention (`torch.nn`),
- obsługi datasetów i data loaderów (`torch.utils.data`),
- akcelerowanych sprzętowo obliczeń na tensorach (na GPU) przy użyciu technologii NVIDIA CUDA w wersji 12.1, co znacznie przyspieszyło proces treningu sieci.

Biblioteka `scikit-learn` posłużyła do:

- preprocessingu danych - skalowania opisanego szczegółowo w poprzednim rozdziale,
- podziału danych na zbiory treningowe, walidacyjne i testowe (funkcja `train_test_split`),
- implementacji modelu bazowego - algorytmu Random Forest,
- obliczania skuteczności modeli, korzystając z dostępnych funkcji (`accuracy_score`, `f1_score`, `roc_auc_score`, `confusion matrix`, itp.).

W celu wizualizacji wyników użyto bibliotek `matplotlib.pyplot` oraz `seaborn` do generowania wykresów strat (ang. *loss curves*), macierzy pomyłek (ang. *confusion matrix*) oraz rozkładu danych.

Do narzędzi pomocniczych zaliczają się:

- `pandas` - do przechowywania i manipulacji danych tabelarycznych (danych wychodzących z potoku przetwarzania),
- `numpy` do operacji na macierzach,

- `joblib` - do serializacji obiektów skalerów, co pozwala na użycie ich w aplikacji webowej opisanej w rozdziale szóstym.

Dla przypomnienia, do budowy potoku przetwarzania obrazu posłużyły biblioteki:

- `ultralytics` - do detekcji twarzy i kadrowania z użyciem YOLOv8,
- `mediapipe` - do ekstrakcji punktów charakterystycznych twarzy,
- `opencv-python` - do operacji na obrazach, w tym: wczytywanie wideo, skalowanie, obliczanie Optical Flow algorytmem Farnebacka,
- `facial_emotion_recognition` - do ekstrakcji wektora emocji,
- `pypmi` - do parsowania plików adnotacji (.eaf) ze zbioru [SDDD](#),
- `tqdm` - do wizualizacji paska postępu podczas długotrwałego procesu przetwarzania danych.

4.2 Model bazowy - Random Forest

Jako model bazowy (*baseline*) zastosowano algorytm Random Forest, którego podstawy teoretyczne zostały dokładnie omówione w rozdziale drugim. Z uwagi na prostotę tego klasyfikatora oraz powszechną dostępność jego implementacji, wykorzystano klasę `RandomForestClassifier` z biblioteki `scikit-learn`.

4.2.1 Przygotowanie danych i wektor wejściowy

W przeciwieństwie do sieci rekurencyjnych, Random Forest nie obsługuje danych sekwencyjnych o zmiennej długości. Ze względu na ten wymóg, sekwencyjne wektory cech musiały zostać spłaszczone do wektorów o stałym rozmiarze, reprezentujących całe nagranie. W tym celu, dla każdej z 23 cech wyznaczono cztery statystyki opisowe: średnia arytmetyczna, odchylenie standardowe, maksimum i zakres wartości. Operacja ta pozwoliła na transformację zmiennej liczby klatek w stały wektor o rozmiarze 92 (23 cechy $\times$ 4 statystyki), który był następnie podawany na wejście klasyfikatora.

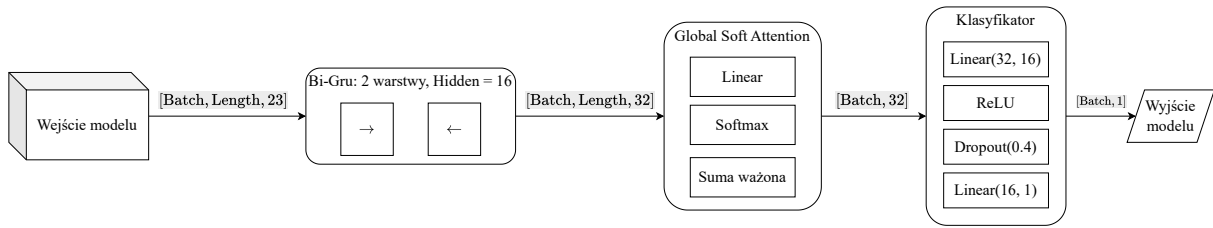
4.2.2 Konfiguracja modelu

Klasa `RandomForestClassifier` jest bardzo konfigurowalna - pozwala na znaczne dostosowanie algorytmu pod swoje własne potrzeby. Hiperparametry zostały dobrane eksperymentalnie, a oto ich wartości:

- Liczba estymatorów (`n_estimators`) = 100: liczba drzew w lesie,
- Maksymalna głębokość drzewa (`max_depth`) = 10: złoty środek między zbyt prostą architekturą (skutkującą niedouczeniem modelu), a zbyt skomplikowaną (mającą ryzyko przeuczenia),
- Ziarno losowości (`random_state`) = 42 (według konwencji): w celu zapewnienia reprodukowalności wyników,
- Waga klas (`class_weight`) = `balanced`: uwzględnienie niebalansowanej liczebności etykiet, wymuszając wagi w funkcji straty odwrotnie proporcjonalne do częstości występowania klas.

4.3 Proponowana architektura głęboka

W tym podrozdziale przedstawione zostaną szczegóły techniczne autorskiego modelu głębokiej sieci rekurencyjnej BiGRU wzbogaconej mechanizmem uwagi.



Rysunek 8. Schemat architektury modelu BiGRU z mechanizmem uwagi. Wymiary tensorów podano w nawiasach kwadratowych nad strzałkami przepływu danych.

4.3.1 Konfiguracja warstwy rekurencyjnej BiGRU

Warstwa wejściowa przyjmuje wektory 23-elementowe (kształt wejścia: `[Batch, Length, 23]`), zgodnie z przedstawionym wcześniej potokiem przetwarzania danych i inżynierią cech. Rozmiar stanu ukrytego został zdefiniowany na 16 jako kompromis między pojemnością informacyjną sieci, a ryzykiem jej przeuczenia na małym zbiorze danych (takim jak **SDDD**, a tym bardziej **RLDDD**). Stan ukryty w każdym kroku czasowym jest konkatencją wektorów z obu kierunków przetwarzania, co opisuje zależność: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$.

Zastosowano dwie warstwy Gru, co zwiększa zdolność modelu do zauważania bardziej złożonych zależności między cechami. Dodatkowo, między warstwami została użyta warstwa **Dropout**. Jej zadaniem jest zapobieganie przeuczeniu. Podczas treningu, w każdym kroku część neuronów (w tym przypadku 40%) jest zerowana. Dzięki temu sieć nie polega na pojedynczych spostrzeżeniach (np. tylko na mruganiu), lecz uczy się bardziej uniwersalnych i odpornych wzorców kłamstwa.

Z powodu nierównej długości próbek został wykorzystany mechanizm paddingu (opisany wcześniej w ostatniej sekcji poprzedniego rozdziału) w postaci funkcji `pack_padded_sequence` i `pad_packed_sequence` z biblioteki `torch.nn.utils`. Pozwala to sieci na przetwarzanie tylko istotnych fragmentów sekwencji, efektywnie pomijając dopełnione wartości, co oszczędza zasoby obliczeniowe i poprawia jakość gradientów w procesie wstecznej propagacji (ang. *backpropagation*).

Wagi rekurencyjne modelu są inicjalizowane stosując inicjalizację ortogonalną. Jest to technika, która matematycznie zapewnia, że wartości własne macierzy wag są bliskie 1. Wpływa to pozytywnie na stabilność uczenia sieci.

4.3.2 Mechanizm uwagi

Zaimplementowano wariant uwagi Global Soft Attention. Warstwa liniowa `torch.nn.Linear` dostaje na wejście skonkatenowany stan ukryty z obu kierunków (ze względu na dwukierunkowość zastosowanej sieci rekurencyjnej BiGRU) i przypisuje mu skalar (score). Następnie na wynikowy wektor (skonkatenowane skalary dla każdej klatki z sekwencji) nakładana jest maska przypisująca klatkom z paddingu wartość `-1e4`. Dzięki użyciu maski po przejściu przez warstwę aktywacji `torch.softmax` ich waga (wartość w finalnym wektorze kontekstu) wynosi dokładnie `0.0`, co sprawia że puste klatki są całkowicie ignorowane. Wagi pozostałych kroków czasowych są równe prawdopodobieństwu wystąpienia w nich istotnej informacji.

Cały ten proces można opisać formalnie w następujących krokach. Niech h_t oznacza wartość stanu ukrytego dla kroku czasowego t . Score obliczany jest ze wzoru:

$$e_t = W^T h_t + b \quad (2)$$

gdzie W i b to trenowalne parametry warstwy liniowej (odpowiednio wagi i bias). Następnie, wagi atencji oznaczone jako α_t obliczane są poprzez normalizację funkcją Softmax:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)} \quad (3)$$

Finalny wektor kontekstu c stanowi sumę ważoną stanów ukrytych wszystkich kroków czasowych z sekwencji:

$$c = \sum_{t=1}^T \alpha_t h_t \quad (4)$$

Tak uzyskany wektor c jest reprezentacją całej sekwencji i trafia do klasyfikatora.

4.3.3 Moduł klasyfikacyjny i wyjście modelu

Wektor kontekstu uzyskany z modułu atencji trafia do perceptrona wielowarstwowego, który składa się z następujących warstw:

- Linear(32, 16),
- ReLU,
- Dropout(0.4),
- Linear(16, 1).

Wąskie gardło (rozmiar 16) przed ostateczną predykcją wymusza na sieci ekstrakcję najważniejszych cech. Model zwraca surowe logity (kształt wyjścia: [Batch, 1]), gdyż wykorzystana podczas treningu funkcja straty `BCEWithLogitsLoss` ma wbudowaną sigmoidę.

4.4 Procedura treningowa i optymalizacja

4.4.1 Funkcja straty i optymalizator

W procesie treningu sieci BiGRU + Attention skorzystano z najbardziej powszechnej funkcji straty używanej do klasyfikacji binarnej, którą jest `BCEWithLogitsLoss`. Funkcja ta łączy w sobie warstwę `Sigmoid` oraz binarną entropię krzyżową (ang. *binary cross entropy*) w jedną operację matematyczną. W przeciwieństwie do ręcznego nakładania funkcji aktywacji na wyjście modelu i korzystania z funkcji straty `BCELoss`, wykorzystana funkcja zapewnia większą stabilność numeryczną, unikając problemów z niedokładnością obliczania gradientów dla skrajnych wartości prawdopodobieństwa.

Zdecydowano się na użycie optymalizatora `AdamW`, który jest poprawioną implementacją klasycznego optymalizatora `Adam`. Wprowadza on bardziej efektywne zanikanie wag (ang. *weight decay*), rozdzielając go od aktualizacji gradientu. Dzięki temu regularyzacja modelu jest silniejsza bez pogarszania zbieżności treningu, co jest kluczowe przy małych zbiorach danych.

4.4.2 Polityka Learning Rate (OneCycle)

Zamiast statycznego rozmiaru kroku lub jego redukcji na płaskim fragmencie wykresu funkcji straty (z użyciem schedulera `ReduceLROnPlateau`), zastosowano politykę **One Cycle Learning Rate**. Trening rozpoczyna się od niższej wartości LR, która rośnie liniowo przez pierwsze 30% epok (tzw. faza warm up), osiągając maksimum na poziomie $1e-3$. W kolejnych iteracjach LR jest sukcesywnie zmniejszane (tzw. faza annealing). Dzięki zastosowaniu schedulera `OneCycleLR`, trening jest szybszy (zjawisko super-zbieżności). Wyższe LR w środkowej fazie działa jak regularyzator, powstrzymując model przed utykaniem w minimach lokalnych.

4.4.3 Inicjalizacja biasu

Dane, na których trenowany jest model są niebalansowane klasowo. Z tego powodu, przed rozpoczęciem treningu, bias ostatniej warstwy klasyfikatora został zainicjowany wartością $\log\left(\frac{\text{num_pos_videos}}{\text{num_neg_videos}}\right)$. Dzięki temu, model od początku przewiduje prawdopodobieństwo zgodne z rozkładem klas w zbiorze treningowym (w którym jest znaczna przewaga próbek zawierających kłamstwo). W ten sposób, początkowa faza treningu skupiona jest już na nauce cech, a nie proporcji klas (jak byłoby w przypadku bez ręcznej inicjalizacji).

4.4.4 Walidacja i wybór metryki

Po zakończeniu każdej epoki treningowej następuje walidacja modelu, podczas której działanie modelu jest ewaluowane na zbiorze walidacyjnym. W tym kroku wagi modelu zostają zamrożone jak przy inferencji, a model traktowany jest jakby był już wytrenowany. Obliczana jest wartość funkcji straty oraz metryki jakościowe, które są akumulowane w celu wygenerowania krzywych uczenia (ang. *learning curves*). Dzięki temu procesowi, możliwe jest badanie jak zmienia się skuteczność modelu oraz wykrywanie zjawiska przeuczenia w czasie rzeczywistym.

Przy niebalansowanym zbiorze danych, model zgadujący zawsze klasę większościową osiąga wysoką skuteczność (accuracy), ale nie odzwierciedla ona rzeczywistej skuteczności klasyfikacji. Z tego powodu, w procesie treningu jako główną metrykę służącą do ewaluacji skuteczności modelu na zbiorze walidacyjnym wybrano pole pod krzywą ROC (AUC ROC, ang. *Area Under the Receiver Operating Characteristic Curve*). Metryka ta mierzy zdolność modelu do rozróżniania klas niezależnie od przyjętego progu klasyfikacyjnego. Wysoka wartość AUC gwarantuje, że model przypisuje wyższe prawdopodobieństwa kłamstwa próbkom fałszywym niż próbkom prawdziwym.

Warto zaznaczyć, że w zadaniach klasyfikacji binarnej, próg rozdzielający klasy o wartości 0.5 nie jest zawsze optymalny. Dlatego, w procesie walidacji stosowana jest technika poszukiwania optymalnego progu (ang. *threshold tuning*), co pozwala na zmaksymalizowanie czułości (ang. *recall* zdefiniowane jako $\frac{\text{true positives}}{\text{all positives}}$) przy zachowaniu akceptowalnego poziomu specyficzności (ang. *specificity* zdefiniowane jako $\frac{\text{true negatives}}{\text{all negatives}}$).

4.4.5 Kontrola pętli treningowej

Proces uczenia realizowany był w trybie wsadowym (ang. *batch training*) z rozmiarem wsadu (ang. *batch size*) równym 16 próbek. Ze względu na rekurencyjny charakter proponowanej sieci neuronowej, zastosowano technikę przycinania gradientu (ang. *gradient clipping*) w celu

eliminacji problemu wybuchających gradientów. Podczas propagacji wstecznej, norma wektora gradientów była skalowana, żeby nie przekraczała wartości 1.0.

Maksymalna liczba epok została ustawiona na 60. Dodatkowo, wykorzystano mechanizm wczesnego zakończenia (ang. *early stopping*) względem metryki AUC z cierpliwością (ang. *patience*) równą 20 epok. Jest to stosunkowo duża cierpliwość, lecz jej wysoka wartość jest kluczowa przy użyciu wcześniej opisanego schedulera `OneCycleLR`. Używając go, strata (jak i inne metryki) może naprzemiennie rosnąć i maleć w fazie wysokiego LR. Z tego powodu, wymagana jest zwiększona tolerancja na brak poprawy, aby dać modelowi szansę na znalezienie lepszego optimum.

Najlepszy model uzyskany podczas trenowania (na podstawie metryki AUC) wczytywany jest na koniec, a jego wagi zapisywane są do pliku w celu możliwości późniejszego go odtworzenia bez konieczności ponawiania treningu.

4.4.6 Podsumowanie hiperparametrów

Finalna konfiguracja hiperparametrów, która pozwoliła na uzyskanie najlepszych wyników (prezentowanych w kolejnym rozdziale), została przedstawiona w poniższej tabeli.

Tabela 1. Zestawienie finalnych hiperparametrów modelu oraz procedury treningowej.

Hiperparametr	Wartość
Architektura sieci	BiGRU + Global Soft Attention
Liczba warstw rekurencyjnych	2
Wymiar stanu ukrytego	16
Dropout	0.4
Optymalizator	AdamW
Funkcja Straty	BCEWithLogitsLoss
Startowe Learning Rate	1e-4
Maksymalne Learning Rate	1e-3 (OneCycle Policy)
Rozmiar wsadu (Batch Size)	16
Liczba epok	60
Regularyzacja L2 (Weight Decay)	0.05
Gradient Clipping Norm	1.0

5 Przeprowadzone eksperymenty i analiza ich wyników

Głównym celem niniejszego rozdziału jest empiryczna weryfikacja skuteczności zaproponowanego rozwiązania BiGRU z mechanizmem uwagi i porównanie jego osiągnięć do modelu bazowego Random Forest. W trosce o maksymalną reprodukowalność eksperymentów ziarna losowości (ang. *random seeds*) bibliotek `torch`, `numpy` i `random` zostały ustalone na stałą wartość, co w połączeniu z udokumentowaną konfiguracją środowiska (wersjami bibliotek) pozwoli innym badaczom na replikację poniżej przedstawionych wyników.

5.1 Spis przeprowadzonych eksperymentów

Oto proces badawczy utworzony z przeprowadzonych eksperymentów:

- 1. Ustawienie punktu odniesienia dla proponowanej architektury sieci głębokiej:** Oddzielny trening klasyfikatora Random Forest na zbiorach **SDDD** oraz **RLDDD**. Głównym celem było wyznaczenie dolnej granicy skuteczności oraz punktu odniesienia dla sieci BiGRU + Attention. Celem pobocznym było wykorzystanie wbudowanej w drzewa decyzyjne analizy ważności cech (ang. *feature importance*), aby zrozumieć, które sygnały są najbardziej informatywne.
- 2. Weryfikacja techniczna (*Overfit Check*):** Przed uruchomieniem pełnego treningu, został przeprowadzony test przeuczenia. Sieć została wytrenowana na pojedynczej paczce danych (16 próbkach) w celu potwierdzenia poprawności architektury modelu oraz zbieżności funkcji straty.
- 3. Ewaluacja modelu autorskiego:** Pełny trening proponowanej sieci na zbiorze **SDDD**. W ramach tego eksperymentu przeprowadzono także analizę krzywych uczenia (ang. *learning curves*), histogramu prawdopodobieństwa oraz macierzy pomyłek wygenerowanych na podstawie inferencji na zbiorze testowym.
- 4. Badanie zdolności generalizacji:** Dotrenowanie modelu na zbiorze danych **RLDDD** z wykorzystaniem techniki transferu wiedzy (ang. *transfer learning*), a w tym także porównanie skuteczności przed (zero-shot) i po dotrenowaniu oraz analiza katastroficznego zapomnienia (ang. *catastrophic forgetting*) na zbiorze oryginalnym.

5.2 Analiza skuteczności modelu bazowego i ważności cech

Wyniki przedstawione w poniższej tabeli zostały uzyskane z modelu Random Forest wytrenowanego na zagregowanych sekwencjach (średnia arytmetyczna, odchylenie standardowe, maksimum i zakres wartości) ze zbiorów **SDDD** i **RLDDD**.

Tabela 2. Porównanie skuteczności modelu na zbiorach Silesian i Real Life

Metryka	SDDD	RLDDD
Accuracy	0.7184	0.7851
F1 Score	0.8343	0.7451
AUC Score	0.6393	0.9063

Wyniki uzyskane na zbiorze **SDDD** są idealnym przykładem modelu Random Forest wytrenowanego na niezbalansowanym zbiorze danych. Stosunkowo wysokie accuracy oraz F1 Score wskazują na dobrze wytrenowany model. Jednak niskie AUC sugeruje, że las losowy ma trudność z rozróżnianiem klas. Model prawdopodobnie nauczył się większości próbek dawać etykietę pozytywną, bo taki jest rozkład danych. Poza problem niezbalansowanych klas, nagrania z tego zbioru pochodzą z laboratorium, co wpływa na zawarte w nich kłamstwa. Niska stawka i brak realnych konsekwencji sprawiają, że sygnały (takie jak m.in. mikroekspresje i wycieki emocjonalne) są słabe i trudne do zauważenia przez algorytm mimo idealnej jakości nagrań.

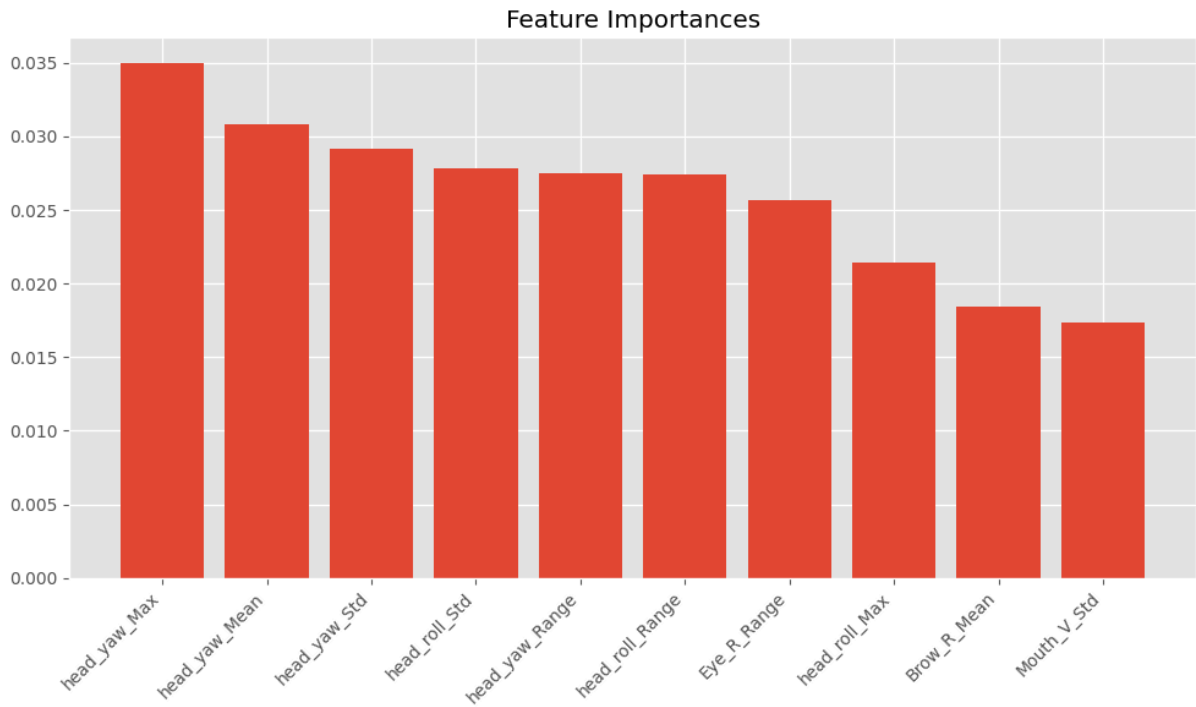
Dla kontrastu, Random Forest poradził sobie znakomicie na zbiorze rzeczywistych nagrań z sal sądowych **RLDDD**, pomimo gorszej ich jakości technicznej. Bardzo wysokie AUC oznacza, że model świetnie separuje prawdę od kłamstwa. Wskazuje to, że w warunkach sali sądowej, gdzie ludzie często kłamiąc walczą o swoją przyszłość, mimowolne reakcje mimiczne i inne zachowania powiązane z kłamstwem są na tyle silne, że przebijają się przez szum słabszej jakości nagrania. Ponieważ zbiór ten jest idealnie zbalansowany, accuracy na poziomie 78,5% jest bardzo dobrym wynikiem dla tak prostego modelu.

Eksperyment sugeruje, że kłamstwo naturalne wiąże się z zupełnie innym zachowaniem niż kłamstwo wymuszone. Random Forest o wiele lepiej radzi sobie z klasyfikacją nagrań zawierających kłamstwa high-stakes, chociaż imbalance klas w zbiorze **SDDD** mógł mocno wpłynąć na działanie modelu wytrenowanego na jego nagraniach.

5.2.1 Analiza ważności cech

Drugim celem eksperymentu z modelem bazowym była analiza ważności cech, która pozwoliła zidentyfikować najbardziej kluczowe behawioralne wyznaczniki kłamstwa. Jak przedstawiono na poniższych wykresach, rankingi najważniejszych cech dla obu zbiorów drastycznie się różnią, co potwierdza fundamentalne różnice w naturze kłamstwa wymuszonego i rzeczywistego.

W przypadku zbioru **RLDDD**, proces podejmowania predykcji został zdominowany przez cechy związane z ruchami głowy (grupa cech **head_pose**), co zostało ukazane na wykresie oznaczonym jako **Rysunek 9**. Aż 6 z 10 najważniejszych cech dotyczy rotacji głowy (**head_yaw** opisujące ruch przeczący „nie” oraz **head_roll**). Istotne okazały się miary zmienności (odchylenie standardowe i zakres wartości). Na podstawie tego można stwierdzić, że w rzeczywistych sytuacjach kłamstwa *high-stakes*, nieszczerości towarzyszą silne, dynamiczne ruchy głowy. Kłamcy prawdopodobnie nieświadomie swoimi ruchami głowy zaprzeczają wypowiedzanym słowom.

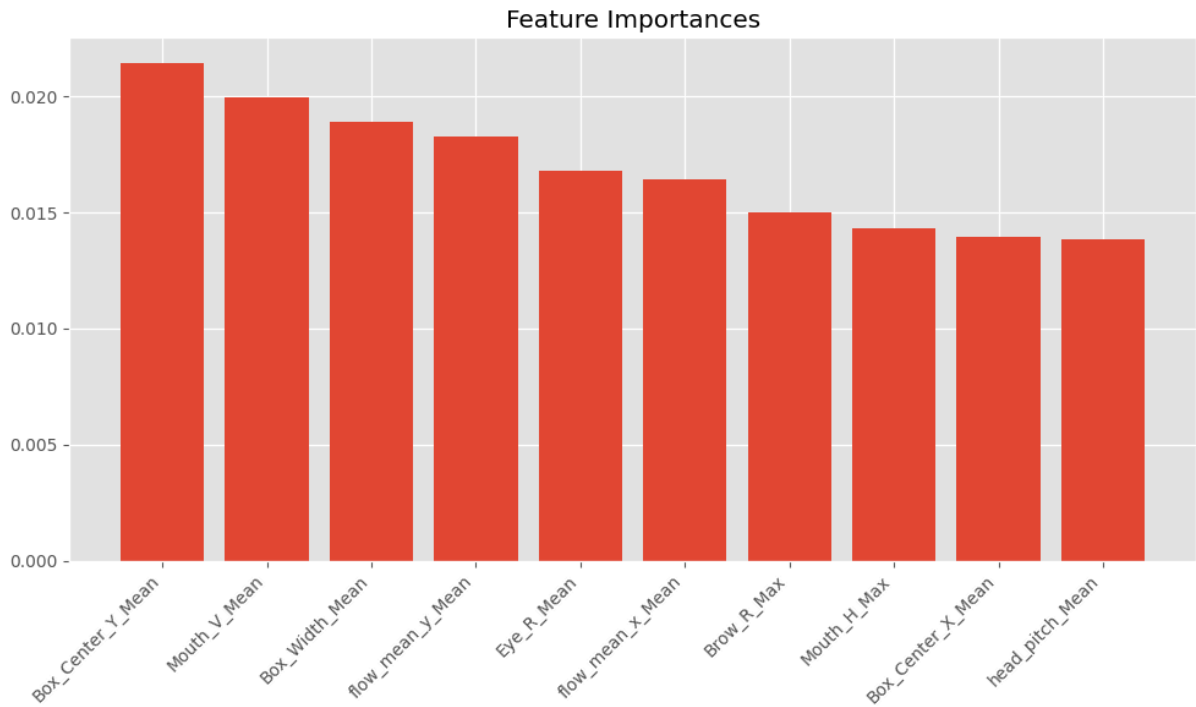


Rysunek 9. Ważność cech: Zbiór RLDDD.

Ważność cech dla zbioru SDDD jest zupełnie odmienna. Tutaj ważności cech z różnych kategorii są mocno przemieszane. W 10 najważniejszych sygnałach znalazły się:

- parametry geometryczne ramki twarzy (`Box_Center_Y`, `Box_Width`) oraz cechy obrazujące ogólny ruch między klatkami, co w przypadku nagrań ze statycznej kamery i z jednolitym tłem wskazuje na zmianę postury ciała - wiercenie się na krześle, pochylanie w przód-tył i ogólnie wysoką ruchowość ciała,
- cechy związane z mimiką (`Mouth`, `Eye` i `Brow`), być może wskazujące mikroekspresje.

Sugeruje to, że w warunkach laboratoryjnych, kłamstwa *low-stakes* nie objawiają się gwałtownymi reakcjami, lecz raczej subtelnymi zmianami pozycji ciała i mikroekspresjami, które są trudniejsze dla prostego modelu do jednoznacznej klasyfikacji, o czym świadczą również wyżej przedstawione metryki.

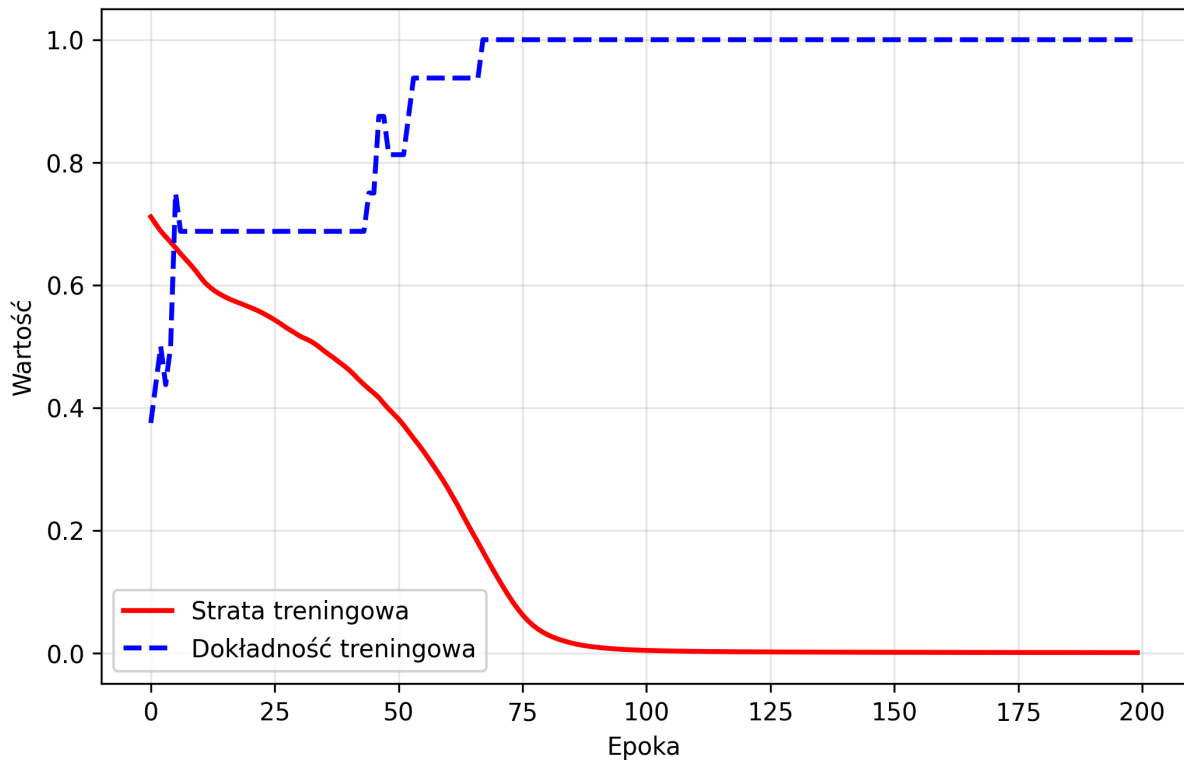


Rysunek 10. Ważność cech: Zbiór SDDD.

5.3 Weryfikacja techniczna modelu autorskiego (*Overfit Check*)

Implementacja złożonych sieci neuronowych jest podatna na błędy, które nie uniemożliwiają uruchomienie kodu, ale uniemożliwiają skuteczny trening modelu. Do takich niedopatrzeń zaliczają się m.in.: złe wymiary warstw, nieprawidłowe połączenia między nimi oraz problemy z paddingiem. Najłatwiejszym sposobem na ich wykrycie jest testowy trening na małym podzbiórze danych. Takie podejście trwa zaledwie kilka sekund i od razu można stwierdzić, czy implementacja modelu jest poprawna.

W tym celu, wytrenowano autorski model BiGRU z mechanizmem uwagi z użyciem pojedynczej paczki danych (*batch*) ze zbioru treningowego, składającej się z 16 próbek. Taki trening powinien doprowadzić do całkowitego przeuczenia. Prawidłowo zaimplementowana architektura posiadająca tysiące trenowalnych parametrów powinna bez żadnego problemu zapamiętać tak małą ilość danych osiągając idealną skuteczność na paczce użytej do treningu.



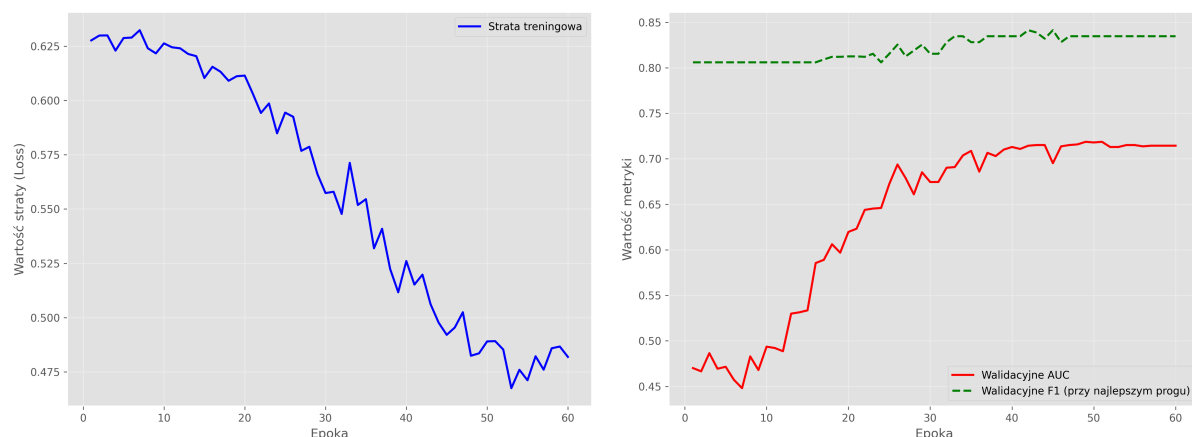
Rysunek 11. Krzywe uczenia uzyskane w procesie *Overfit Check*

Jak przedstawiono na powyższej krzywej uczenia, model bardzo szybko zredukował stratę do wartości bliskich zero (w 100 epok). Wszystkie próbki zostały bezbłędnie sklasyfikowane, o czym świadczy *accuracy* równe 1.0. Dzięki temu, potwierdzono, że dane poprawnie płyną przez wszystkie warstwy sieci, a gradienty są precyzyjnie obliczane i aktualizują wagi w przewidywany sposób. Oznacza to także, że architektura ma wystarczającą pojemność informacyjną, żeby nauczyć się zależności między cechami a etykietami. Pozytywny wynik testu pozwolił na rozpoczęcie właściwego procesu uczenia na pełnym zbiorze treningowym.

5.4 Analiza procesu uczenia i skuteczności modelu autorskiego

Przedstawione w tym podrozdziale wyniki zostały uzyskane podczas treningu autorskiego modelu BiGRU z mechanizmem uwagi na zbiorze SDDD z użyciem hiperparametrów przedstawionych w sekcji zamykającej rozdział 4.

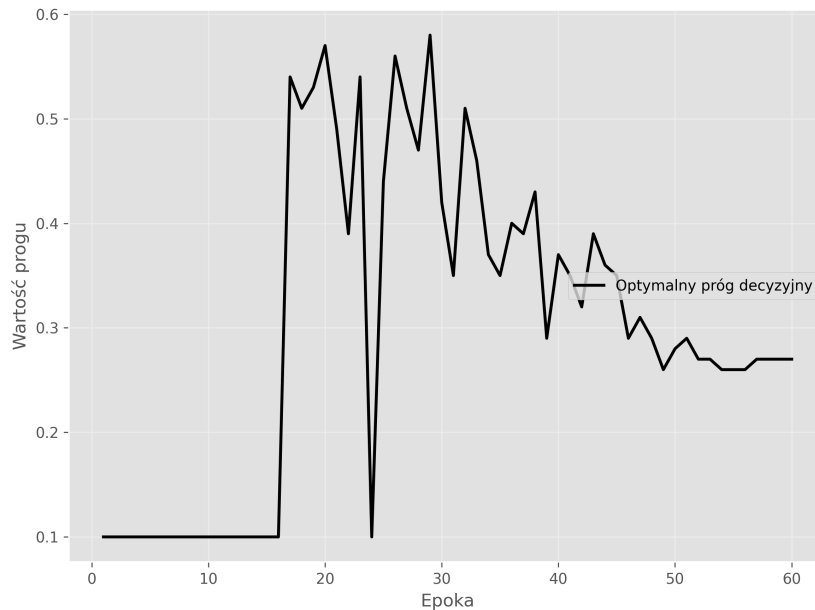
Celem analizy przebiegu procesu uczenia jest weryfikacja poprawności doboru hiperparametrów, skuteczności polityki treningowej *OneCycleLR* oraz ocena zdolności modelu do generalizacji wiedzy. Po każdej epoce monitorowane były metryki jakościowe, co pozwoliło na ocenę stabilności zbieżności algorytmu optymalizacyjnego. Poniższa sekcja prezentuje analizę wartości tych metryk w czasie treningu oraz finalną ocenę skuteczności modelu na zbiorze testowym.

5.4.1 Dynamika procesu uczenia (*Learning Curves*)

Rysunek 12. Krzywe uczenia dla modelu BiGRU na zbiorze SDDD. Po lewej: przebieg funkcji straty (BCEWithLogitsLoss). Po prawej: przebieg metryki AUC ROC oraz F1 Score.

Na wykresie funkcji straty widoczny jest wpływ polityki `OneCycleLR`. Na początku, niskie startowe LR powoduje wolny spadek wartości funkcji straty, po czym następuje przyspieszenie (faza *warm-up*). Pod koniec treningu widoczne jest ponowne spowolnienie, ponieważ w fazie *annealing* długość kroku jest redukowana. Podczas całego procesu uczenia, wartość funkcji straty nie spadła blisko zera, co oznacza, że maksymalna wartość hiperparametru *learning rate* została dobrana prawidłowo i nie wystąpiło zjawisko znacznego przeuczenia. Wahania widoczne na krzywej prawdopodobnie są spowodowane stosunkowo niskim rozmiarem paczki danych, ale nie powinno to negatywnie wpłynąć na skuteczność modelu.

Wysoka początkowa wartość F1 Score na zbiorze walidacyjnym (ok. 0.81) spowodowana została przez imbalans klas i inicjalizację wag biasu klasyfikatora. Niemniej jednak, wartość tej metryki powoli wzrasta (głównie w środkowej fazie treningu za sprawą użytej polityki treningowej), osiągając maksimum bliskie wartości 0.85. Pole pod krzywą ROC na początku treningu oscylowało w okolicach losowości - 0.5. Oznacza to, że model w początkowej fazie nauki nie rozróżniał jeszcze klas. Jednak, po ok. 15 epokach, ta wartość zaczęła stosunkowo stabilnie rosnąć, dochodząc aż do prawie 0.72. Tak wysoka wartość sugeruje, że wytrenowany już model faktycznie nauczył się separować kłamstwa od szczerych wypowiedzi.



Rysunek 13. Przebieg optymalnego progu decyzyjnego.

Z analizy zmienności optymalnego progu decyzyjnego przedstawionej na powyższym wykresie można także wyciągnąć interesujące wnioski. Przez pierwsze 15 epok wykres zmian progu decyzyjnego jest płaski i trzyma się minimalnej dopuszczanej wartości 0.1. Idealnie pokrywa się to z wykresem AUC, który w tym czasie wskazywał na losowe zgadywanie. Model w tej fazie dopiero „się rozgrzewał”, ucząc się czym są poszczególne cechy, ale jeszcze nie zauważając zależności między nimi.

Gdy AUC zaczęło rosnąć (model zaczyna dostrzegać różnice między klasami), optymalny próg decyzyjny skakał między wartościami 0.1 i 0.6. Jest to efekt wysokiego LR w tej fazie treningu. Od epoki 45, wahania zaczęły się wygaszać, a próg ustabilizował się na poziomie ok. 0.27. Model w tym momencie zakończył już naukę cech i tylko delikatnie się dostrajał. Finalny próg jest znacznie niższy od domyślnego 0.5 dowodząc niezbędności zastosowania techniki *threshold tuning*. Ze względu na niezbalansowanie zbioru i specyfikę funkcji straty, model jest „nieśmiały” w swoich predykcjach. Sztywne przyjęcie domyślnego progu skutkowałoby nieoptymalnymi finalnymi wartościami metryk F1 Score i Accuracy.

5.4.2 Wyniki na zbiorze testowym i porównanie z modelem bazowym

W tabeli poniżej przedstawiono wyniki opisujące dokładność predykcji najlepszego uzyskanego modelu (na podstawie metryki AUC) z epoki 51 na zbiorze testowym. Dla porównania w tabeli umieszczono także wyniki osiągnięte przez model bazowy Random Forest.

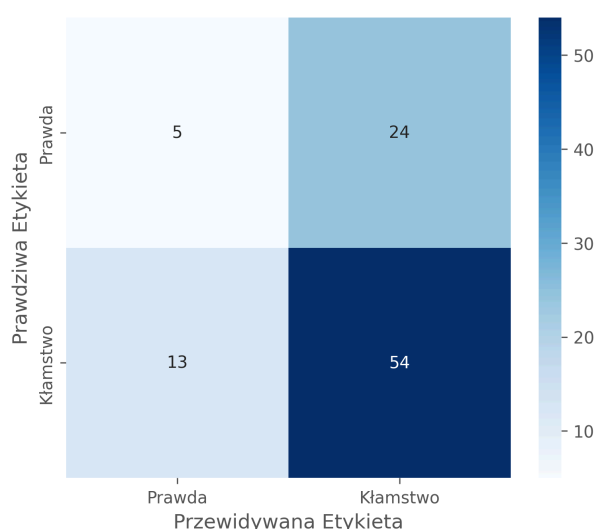
Tabela 3. Zestawienie wyników końcowych modelu autorskiego z modelem bazowym na zbiorze testowym Silesian.

Metryka	Random Forest (Baseline)	BiGRU + Attention
Accuracy	0.7184	0.6146
F1 Score	0.8343	0.7448
AUC ROC	0.6393	0.5234

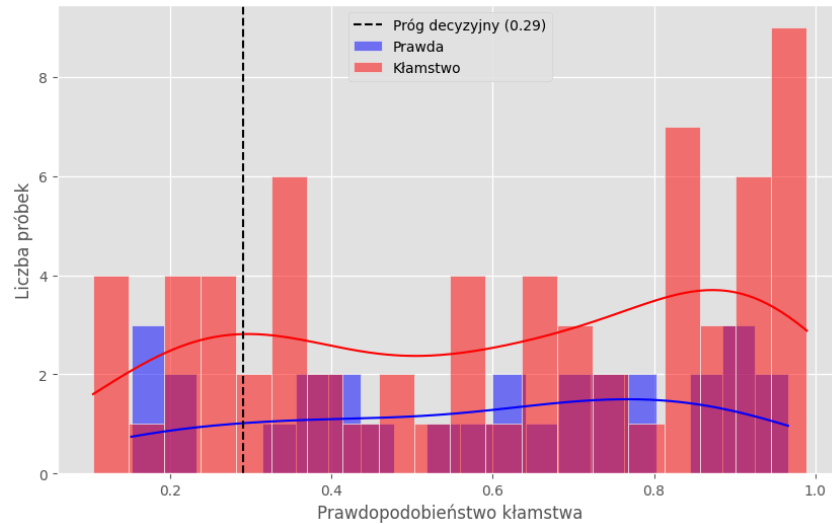
Mimo obiecujących wyników na etapie walidacji (AUC równe ok. 0.72), finalny model poradził sobie z inferencją na zbiorze testowym znacznie gorzej, a także niestety gorzej niż model bazowy. W kontraście do wyników walidacyjnych, na zbiorze testowym AUC spadło do poziomu zaledwie 0.52 - wynik zbliżony do losowego zgadywania. Oznacza to, że model dopasował się do specyfiki osób ze zbioru walidacyjnego, ale nie nauczył się w pełni rozpoznawać uniwersalnych wskaźników kłamstwa. Wskazuje to na trudność w generalizacji wyuczonych wzorców na nowe osoby, mimo zastosowania technik regularyzacji w postaci warstw **Dropout** oraz mechanizmu zanikania wag (*weight decay*).

5.4.3 Analiza macierzy pomyłek (*Confusion Matrix*) i histogramu prawdopodobieństwa

W celu zbadania nietypowego zjawiska wysokiego F1 Score przy tak niskim AUC wygenerowano macierz pomyłek widoczną poniżej. Aż 80.6% kłamców zostało sklasyfikowanych poprawnie, ale 82% osób mówiących prawdę także zostało oskarżone o nieszczerłość. Model wykazuje silną tendencję do klasyfikowania próbek jako kłamstwo w przypadku niepewności (tzw. bias w stronę klasy większościowej). Nie znajdując silnych sygnałów w danych, przyjmuje on bezpieczną strategię obstawiania klasy większościowej (kłamstwa).

**Rysunek 14.** Macierz pomyłek wytrenowanego modelu BiGRU+Attention na podzbiorze testowym zbioru SDDDD.

Potwierdza to także histogram, na którym rozkłady prawdopodobieństw dla prawdy i kłamstwa w dużym stopniu na siebie nachodzą, uniemożliwiając wyznaczenie skutecznej granicy decyzyjnej.



Rysunek 15. Histogram prawdopodobieństw predykcji modelu BiGRU+Attention na podzbiorze testowym zbioru SDDD.

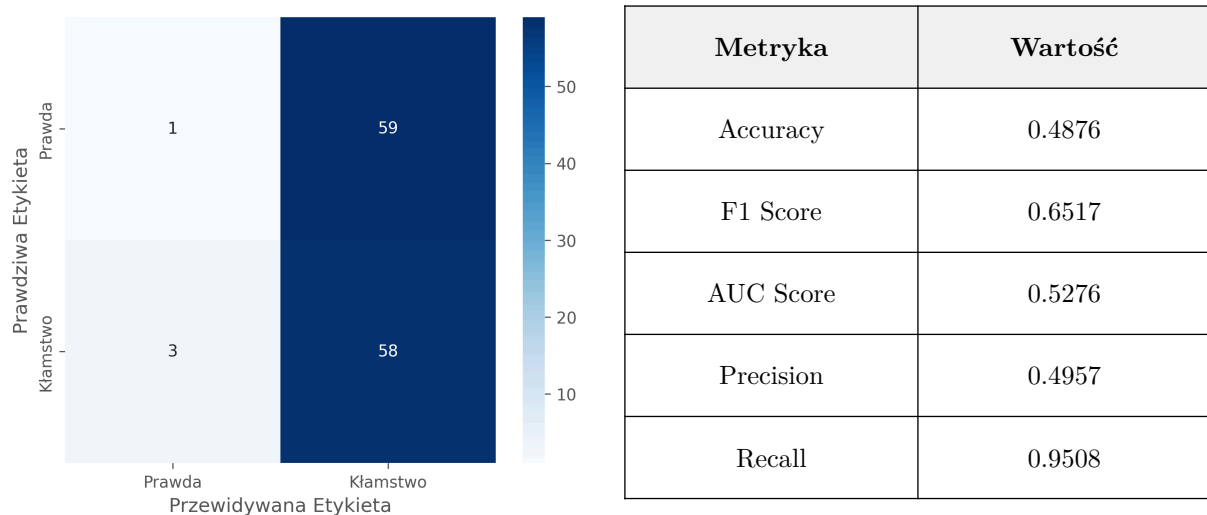
Wnioskiem z eksperymentu jest to, że proste modele działające na zagregowanych statystykach cech (Random Forest) radzą sobie lepiej niż złożone modele sekwencyjne. Prawdopodobnie, zostało to spowodowane niską liczebnością zbioru SDDD, która okazała się niewystarczająca do wytrenowania tysięcy parametrów tak rozbudowanego modelu głębokiego.

5.5 Transfer wiedzy (ang. *transfer learning*)

Celem tego podrozdziału jest głębsze zbadanie zdolności generalizacji modelu i weryfikacja hipotezy o negatywnym wpływie laboratoryjnych wymuszonych kłamstw low-stakes na skuteczność predykcji. Przeprowadzono transfer wiedzy modelu wytrenowanego na zbiorze SDDD, dotrenowując go na RLDDD.

5.5.1 Ewaluacja zero-shot

Na początku, wykonano tzw. ewaluację zero-shot - sprawdzenie skuteczności wytrenowanego już modelu na odmiennym zbiorze testowym (pochodzącym z RLDDD). Wyniki zostały zamieszczone w poniższej tabeli oraz macierzy pomyłek.

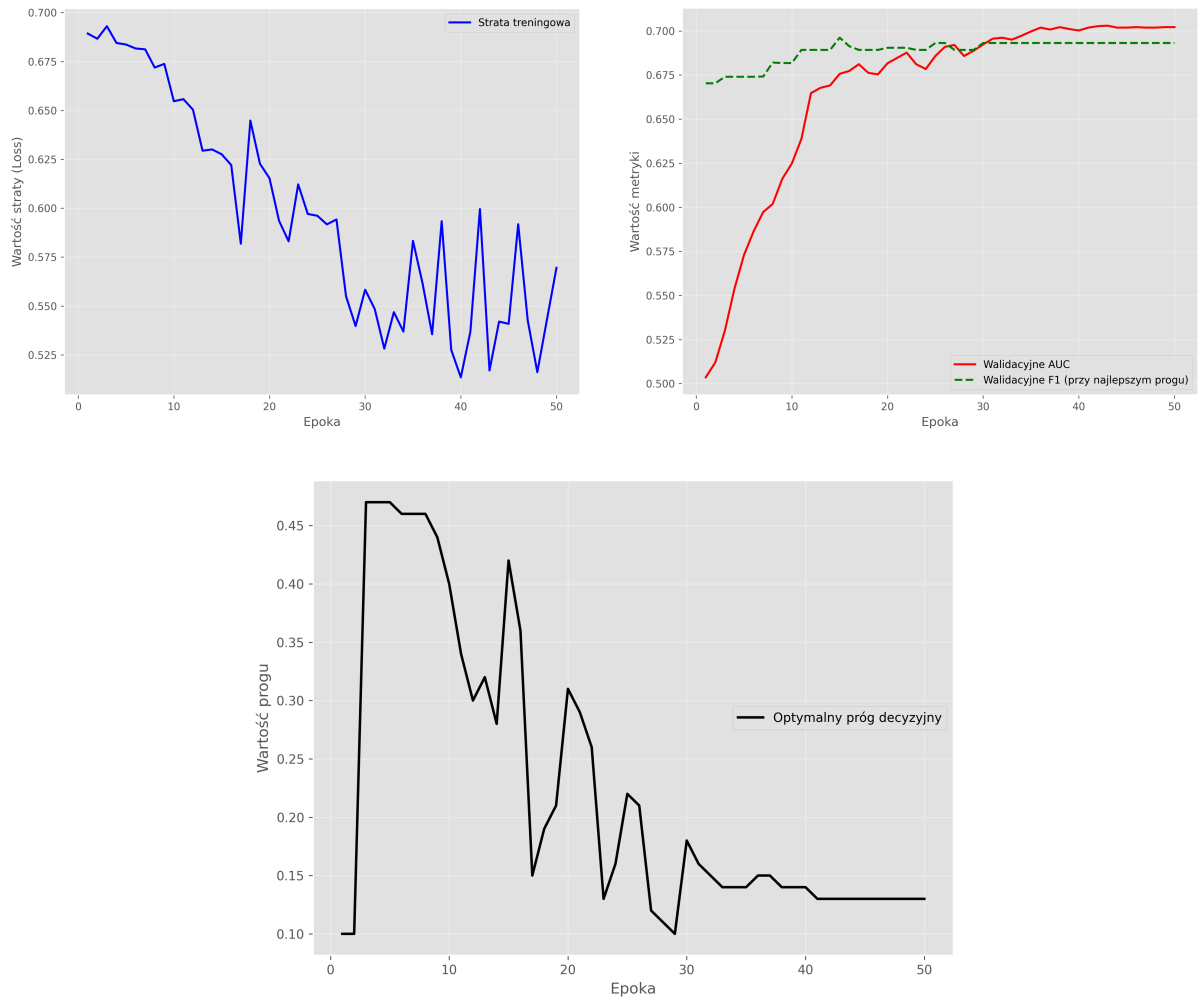


Rysunek 16. Statystyki uzyskane z ewaluacji zero-shot na zbiorze RLDDD. Po lewej: macierz pomyłek. Po prawej: metryki jakościowe.

Jak można było się spodziewać, model nie osiągnął zadowalających wyników. Podobnie jak w przypadku ewaluacji na zbiorze testowym z SDDD, przewidywał on kłamstwo dla większości próbek. Wartość AUC jest porównywalna z tą uzyskanym na podzbiorze testowym zbioru oryginalnego, a niższe accuracy w tym przypadku można wytłumaczyć równomiernym rozkładem etykiet. Ewaluacja zero-shot wskazuje na to, że model wytrenowany na danych laboratoryjnych nie jest dobrym klasyfikatorem dla danych rzeczywistych bez ówczesnego dotrenowania.

5.5.2 Procedura transferu wiedzy i analiza jego wyników

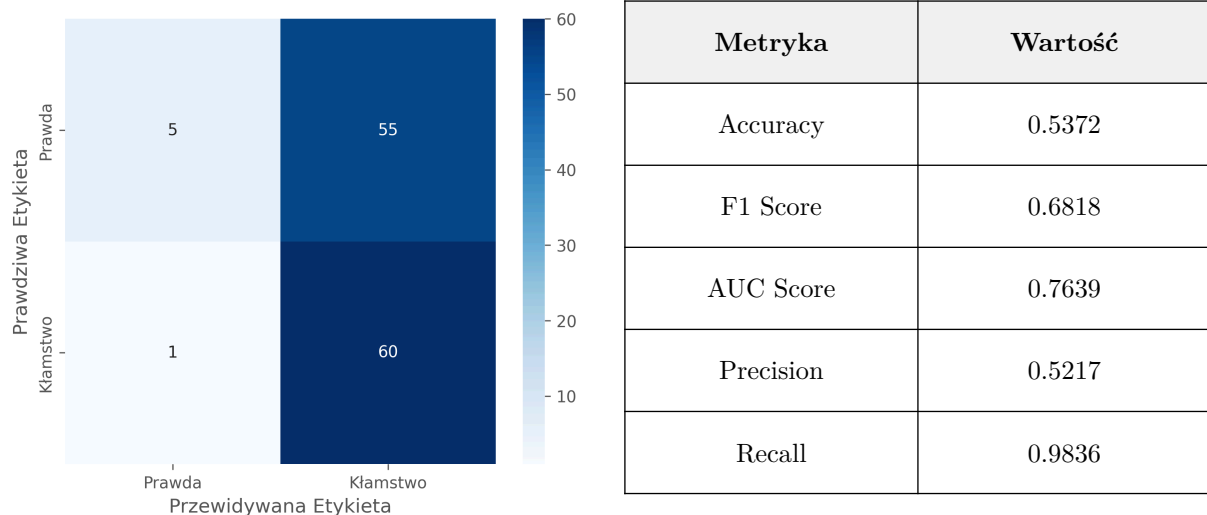
Zamiast uczyć model od zera, zastosowano technikę transfer learningu. Zamrożono warstwy BiGRU oraz uwagi, aby zachować wytrenowaną już ekstrakcję cech. Zdecydowano się na użycie nowej „wyzerowanej” głowy klasyfikacyjnej, której bias został zainicjowany w taki sam sposób jak przy oryginalnym treningu (logarytm stosunku klas). Ponownie, użyto schedulera OneCycleLR. Miało to na celu ułatwienie i przyspieszenie treningu. Poniżej przedstawione zostały krzywe uczenia.



Rysunek 17. Dynamika procesu transfer learningu modelu autorskiego na zbiorze RLDDD. W górnym rzędzie: przebieg funkcji straty (po lewej) oraz metryk walidacyjnych AUC i F1 (po prawej). U dołu: ewolucja optymalnego progu decyzyjnego w czasie trwania treningu.

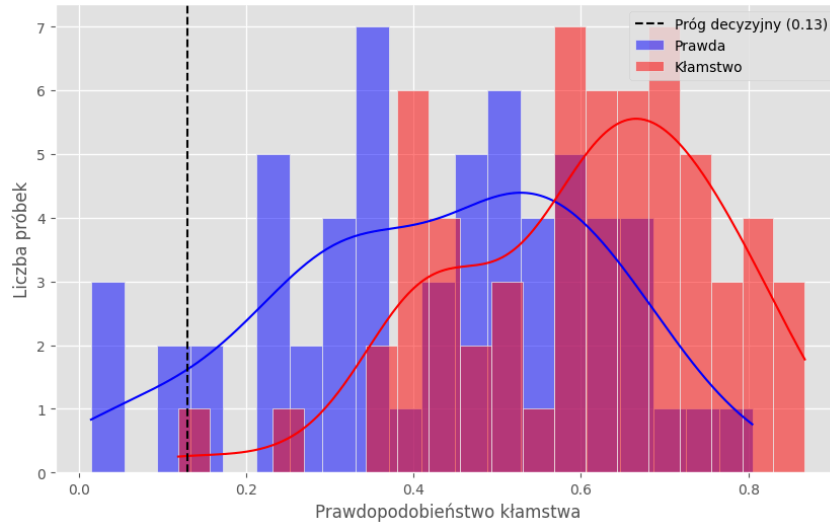
Wartość funkcji straty stabilnie spada (pomijając mocne wahania w ostatniej fazie treningu - prawdopodobnie za wysokie LR na trening wyłącznie warstw klasyfikacyjnych), a AUC bardzo szybko rośnie do poziomu ok. 0.7. Oznacza to, że cechy wyekstrahowane przez BiGRU na zbiorze SDDD są użyteczne. Wagi sieci rekurencyjnej są poprawnie wytrenowane, dzięki czemu model rozumie ruchy twarzy, a sam klasyfikator musiał tylko nauczyć się podejmować decyzję na ich podstawie.

Wyniki finalnej ewaluacji dotrenowanego modelu na zbiorze testowym z RLDDD zostały przedstawione poniżej.



Rysunek 18. Statystyki uzyskane z ewaluacji dotrenowanego modelu na zbiorze testowym z RLDDD. Po lewej: macierz pomyłek. Po prawej: metryki jakościowe.

Osiągnięte AUC przewyższyło to uzyskane na oryginalnym zbiorze danych (SDDD), ale jest niższe niż to uzyskane przez Random Forest. Dowodzi to, że wskaźniki kłamstwa są znacznie wyraźniejsze w sytuacjach *high-stakes* (sala sądowa) niż w *low-stakes* (laboratorium). Zamrożony ekstraktor cech (BiGRU) skutecznie wykrywa te sygnały, a nowa głowa klasyfikacyjna nauczyła się je interpretować. Warto jednak, zwrócić uwagę na niską wartość Accuracy przy bardzo wysokim Recall. Prawdopodobnie wynika to z faktu, że w warunkach sądowych stres, jak i obciążenie poznawcze towarzyszy zarówno kłamcom, jak i osobom prawdomównym. Model wykrywając silne napięcie, klasyfikuje większość osób jako podejrzane, z czego wynika także niski próg decyzyjny (0.13). Mimo to, wysokie AUC dowodzi, że model poprawnie nadaje wyższe prawdopodobieństwo kłamstwa rzeczywistym kłamcom, co jest widoczne także na poniższym histogramie, gdzie rozkłady prawdopodobieństw klas są od siebie lepiej odseparowane niż w przypadku wyników na oryginalnym zbiorze.



Rysunek 19. Histogram prawdopodobieństw predykcji dotrenowanego modelu BiGRU+Attention na podzbiorze testowym zbioru **RLDDD**.

5.5.3 Analiza katastroficznego zapomnienia

Jako finalny eksperyment związany z techniką transferu wiedzy, postanowiono sprawdzić czy skuteczność modelu dotrenowany na nagraniach sądowych poprawiła się względem oryginalnie wytrenowanego modelu. Oryginalny klasyfikator na zbiorze testowym **SDDD** miał AUC równe 0.5279, a po dotrenowaniu wartość tej metryki spadła do 0.4731. Pomimo wysokiej wartości tego wskaźnika na zbiorze testowym **RLDDD**, wynik spadł tutaj poniżej losowości. Model nie tylko zapomniał, ale zaczął wręcz mylić sygnały ze starego zbioru.

W **SDDD** nie ma silnego stresu. Nowa głowa klasyfikacyjna szuka w nagraniach silnych reakcji na stres, ale nie udaje się jej ich znaleźć, przez co produkuje chaotyczne predykcje. Stara głowa natomiast, potrafiła lepiej skupić się na subtelniejszych sygnałach i wykonywać nieco bardziej precyzyjne predykcje. Eksperyment dowodzi, że nie da się skonstruować jednego modelu do klasyfikacji zarówno kłamstw *low-stakes* i *high-stakes*, gdyż różnica między tymi domenami (ang. *domain gap*) jest zbyt duża.

6 Implementacja systemu w postaci aplikacji webowej

Nawet najbardziej skuteczne modele uczenia maszynowego są bezużyteczne w zastosowaniach komercyjnych, jeśli są dostępne wyłącznie jako skrypt uruchamiany z wiersza poleceń lub komórka w Jupyter Notebooku. Celem niniejszego rozdziału jest prezentacja interfejsu graficznego przygotowanego dla opracowanych modeli umożliwiającego ich użycie przez użytkownika końcowego, nieposiadającego wiedzy programistycznej. Stworzone rozwiązanie ma charakter Proof of Concept (PoC). Jest to prototyp mający na celu demonstrację możliwości utworzonego systemu automatycznej analizy prawdomówności, a nie gotowy produkt komercyjny.

Treść tego rozdziału rozpocznie się od definicji wymagań funkcjonalnych i нефункциональных. Następnie, omówiony zostanie stos technologiczny wraz z argumentacją za jego wyborem, opis architektury. Na końcu, zaprezentowane będzie działanie stworzonego prototypu.

6.1 Analiza wymagań systemowych

Zgodnie z zasadami inżynierii oprogramowania, proces budowy aplikacji poprzedzono definicją wymagań. Ze względu na badawczy charakter projektu (*Proof of Concept*), skupiono się na kluczowych funkcjonalnościach niezbędnych do walidacji modeli w środowisku produkcyjnym. Wymagania podzielono na dwie kategorie: funkcjonalne (określające zachowanie systemu) oraz нефункциональные (określające atrybuty jakościowe).

6.1.1 Wymagania funkcjonalne

System musi realizować następujące funkcje:

- **WF1. Obsługa danych wejściowych:** Możliwość wczytania pliku wideo w powszechnych formatach (m.in. .mp4, .avi) poprzez interfejs graficzny.
- **WF2. Konfiguracja inferencji:** Możliwość dynamicznego wyboru architektury modelu (Random Forest, BiGRU) oraz zbioru danych, na którym został wytrenowany (Silesian/Real-Life).
- **WF3. Automatyczne przetwarzanie nagrań:** System musi automatycznie wykonywać potok przetwarzania wideo w dane zrozumiałe dla modeli bez ingerencji użytkownika.
- **WF4. Prezentacja wyników:** Aplikacja musi wyświetlać wynik predykcji w przejrzysty i zrozumiały sposób - binarnie i probabilistycznie.
- **WF5. Wyjaśnialność (XAI):** Dla modeli głębokich system musi wyświetlać wykres wag uwagi w funkcji czasu.
- **WF6. Responsywność:** System musi na bieżąco informować o postępie analizy i wystąpieniach błędów.

6.1.2 Wymagania нефункциональные

System musi spełniać następujące kryteria jakościowe:

- **WN1. Użyteczność:** Interfejs użytkownika nie może wymagać wiedzy programistycznej ani technicznej i powinien być zrozumiały dla laika.
- **WN2. Wydajność:** Czas przetwarzania powinien być zminimalizowany z użyciem downsamplingu.

- **WN3. Zarządzanie zasobami:** Wagi modeli powinny być ładowane do pamięci operacyjnej tylko raz, aby przełączanie między nagraniami było płynne.
- **WN4. Odporność na błędy użytkownika:** Aplikacja musi bezpiecznie obsługiwać wyjątki, takie jak brak twarzy w nagraniu, wyświetlając odpowiedni komunikat zamiast przerywać działanie.
- **WN5. Przenośność:** System powinien być niezależny od systemu operacyjnego oraz podzespołów komputera, działając zarówno na CPU jak i GPU.

6.2 Założenia projektowe i wybór technologii

Projektując interfejs użytkownika postawiono na jego prostotę i czytelność. Aplikacja ta powinna działać na zasadzie „drag and drop”, minimalizując wymaganą liczbę kliknięć potrzebnych do otrzymania predykcji dla własnego nagrania. Ze względu na uzyskane wyniki opisane w poprzednim rozdziale (bardzo odmienne skuteczności modelu autorskiego BiGRU+Attention wytrenowanego na zbiorze **SDDD**, dotrenowanego na **RLDDD**, a także obu klasyfikatorów Random Forest) zdecydowano się na podejście wielomodelowe. System musi pozwalać na dynamiczne przełączanie się między różnymi klasyfikatorami bez konieczności restartowania aplikacji. Dodatkowym założeniem była także optymalizacja procesu inferencji w celu jak najszybszego czasu analizy, co uzyskano poprzez m.in. inteligentne pomijanie klatek (*downsampling*) oraz akcelerację obliczeń na karcie graficznej (w przypadku jej dostępności).

Poniżej przedstawiono wykorzystany stos technologiczny:

- Język programowania: **Python**. Jest to standard współczesnego uczenia maszynowego. Użycie tego języka pozwoliło na bezpośrednią integrację kodu napisanego w fazie badań i modelowania oraz wczytanie wag najlepszych modeli zapisanych w trakcie treningu.
- Framework UI: **Streamlit**. Został wybrany zamiast klasycznego podejścia dwuwarstwowego z oddzielnym backendem i frontendem, ponieważ pozwala na błyskawiczne tworzenie aplikacji opartych na danych w czystym Pythonie.
- Silnik uczenia maszynowego: **PyTorch i Scikit-learn**.
 - PyTorch został użyty do inferencji z wykorzystaniem modelu głębokiego (BiGRU+Attention).
 - Scikit-learn został wykorzystany do obsługi modeli Random Forest.
- Przetwarzanie danych: **OpenCV, MediaPipe, FacialEmotionRecognition, Ultralytics**. Wykorzystano odtworzony potok opisany szczegółowo w rozdziale trzecim tej pracy. Dostosowano go, jednak, do przetwarzania pojedynczych nagrań w przeciwieństwie do pierwotnego zastosowania na całych zbiorach danych.

Taki dobór technologii jest obecnie standardem w szybkim prototypowaniu rozwiązań AI w środowiskach R&D (ang. *research and development*).

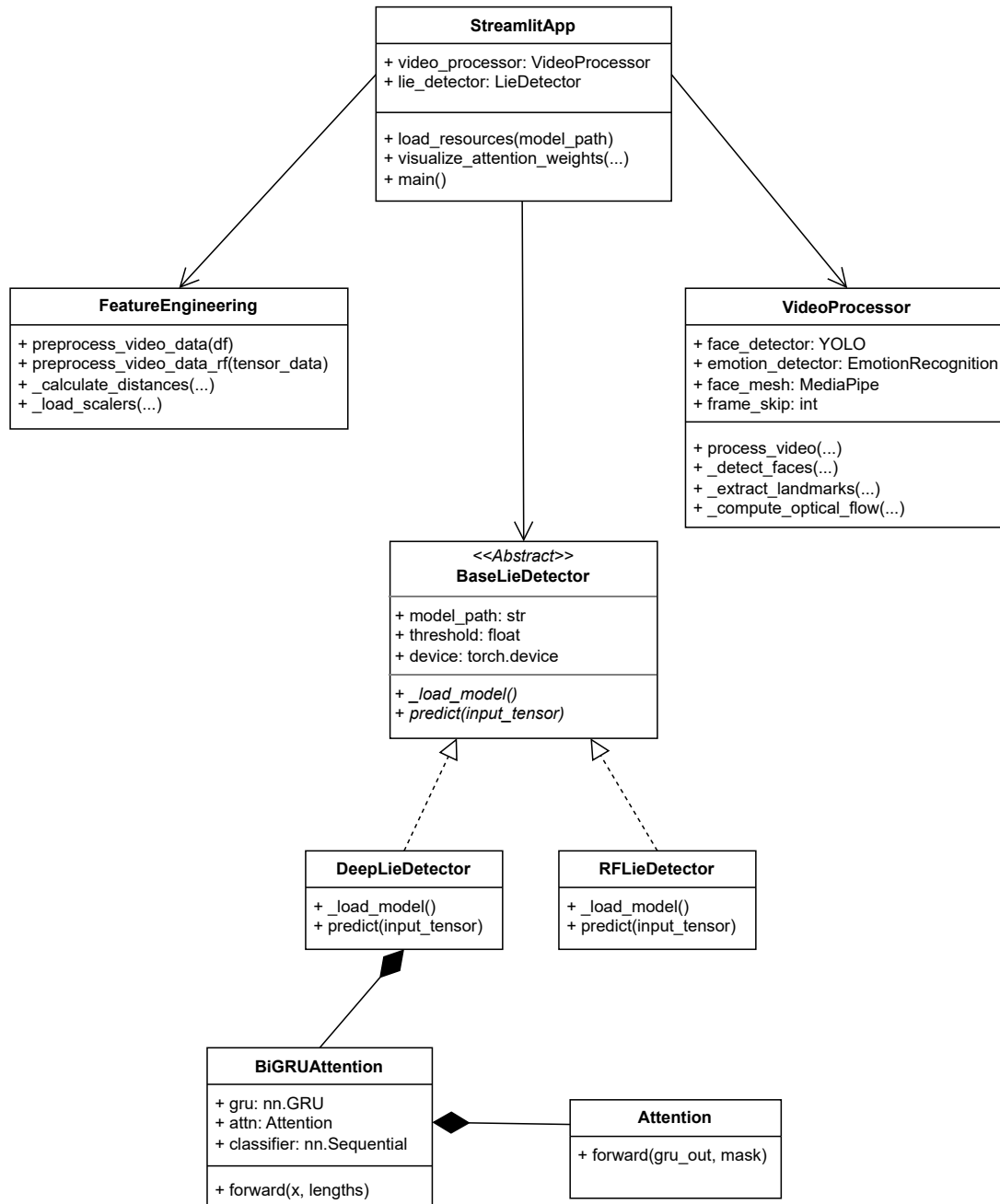
6.3 Architektura aplikacji

W procesie projektowania systemu skupiono się na modularności i separacji odpowiedzialności, co ma na celu zachowanie czytelności kodu oraz łatwość rozszerzania go w przyszłości o dodatkowe funkcjonalności. Logika aplikacji została podzielona na trzy warstwy, separując logikę interfejsu graficznego, przetwarzania danych i inferencji.

Sercem systemu jest moduł predykcji. Aby obsłużyć dwa zupełnie różne typy modeli (sieci neuronowe i lasy losowe) w jednolity sposób, zdefiniowano abstrakcyjną klasę bazową **BaseLieDetector** oraz fabrykę **LieDetector**, która na podstawie rozszerzenia wybranego przez użytkownika pliku wag dynamicznie instancjonuje odpowiedni obiekt klasyfikatora. Dzięki zastosowaniu polimorfizmu, warstwa interfejsu nie uwzględnia szczegółów implementacyjnych modelu, a jedynie wywołuje ustandaryzowaną metodę **predict**.

Za wstępne przetworzenie nagrań i konwersję do formy akceptowanej przez modele odpowiadają moduły pomocnicze. Klasa **VideoProcessor** dokładnie odtwarza logikę potoku przetwarzania obrazu wykonywanego przed trenowaniem modeli. Funkcje zdefiniowane w pliku **features.py** wyciągają inteligentne (ang. feature-engineered) wskaźniki, obliczając znormalizowane dystanse między punktami charakterystycznymi twarzy, konstruując cechy pochodne (prędkościowe), aplikując filtry wygładzające (średnia krocząca) i odpowiednio skalując dane.

Logika samego interfejsu graficznego zaimplementowana została w pliku **app.py**. Moduł ten pełni rolę orkiestratora systemu - definiuje układ komponentów wizualnych, obsługuje interakcję z użytkownikiem, wywołuje odpowiednie funkcje pomocnicze i predykcje modeli, a także odpowiada za wizualizację wyników. Ważnym aspektem architektury jest optymalizacja wydajności. Wykorzystano mechanizm cache'owania, aby zapobiec wielokrotnemu wczytywaniu wag modeli do pamięci operacyjnej. Ładowanie to jest wykonywane tylko raz (przy uruchomieniu strony) oraz po każdej zmianie wybranego modelu.



Rysunek 20. Diagram klas UML prezentujący architekturę aplikacji. Widoczny podział na warstwę interfejsu (*StreamlitApp*), przetwarzania wideo (*VideoProcessor*) oraz hierarchię klas modułu predykcyjnego realizującą polimorfizm.

6.4 Weryfikacja działania i testowanie oprogramowania

Zapewnienie niezawodności systemu łączącego przetwarzanie wideo, modele sztucznej inteligencji oraz interfejs użytkownika wymagało wprowadzenia procedury testowej. W procesie jej implementacji wykorzystano framework *pytest*, który jest standardem w inżynierii oprogramowania z użyciem języka Python. Kluczowym elementem strategii testowania było wykorzystanie techniki *atrap obiektów* (ang. *mocking*) z biblioteki *unittest.mock*, co pozwoliło na oddzielenie logiki biznesowej od ciężkich zasobów (w tym wag modeli, operacji wejścia i wyjścia).

6.4.1 Testy jednostkowe

W celu weryfikacji poprawności działania pojedynczych, izolowanych komponentów utworzono testy jednostkowe:

- **Weryfikacja inżynierii cech:** Przetestowano kompletny potok przetwarzania danych dla obu typów modeli. Dla modelu głębokiego sprawdzono czy funkcja `preprocess_video_data()` generuje odpowiednie sekwencyjne tensory. Dla modelu Random Forest zweryfikowano funkcję `preprocess_video_data_rf()` pod kątem poprawnej transformacji sekwencji czasowej w „spłaszczony” wektor o stałym wymiarze 92 cech. Dodatkowo, przetestowano funkcję `calculate_distances()`.
- **Walidacja architektury sieci:** Sprawdzono poprawność implementacji klasy `BiGRUAttention`, weryfikując zgodność wymiarów tensorów wejściowych i wyjściowych oraz poprawność działania mechanizmu uwagi (suma wag powinna równać się 1.0 po operacji Softmax).

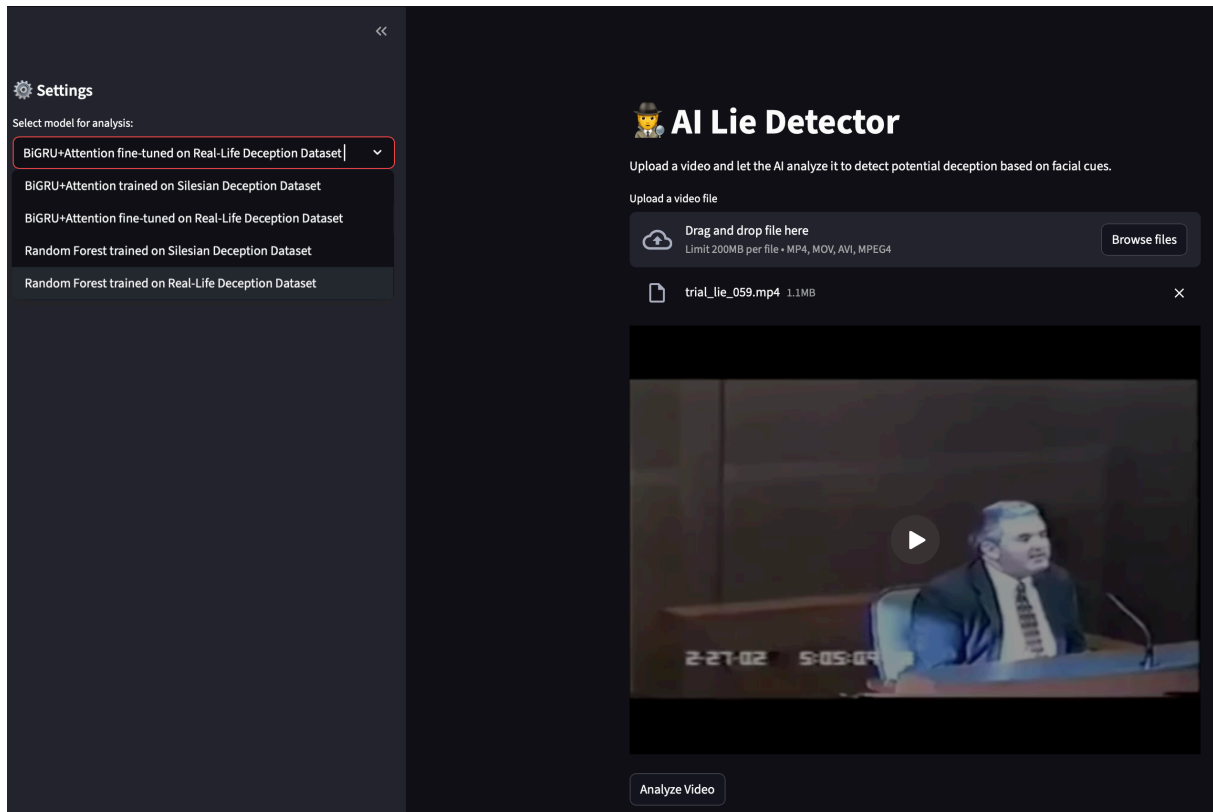
6.4.2 Testy integracyjne

Celem testów integracyjnych było sprawdzenie poprawności współpracy między modułami w ramach potoku predykcyjnego, odzwierciedlającego ten proces w aplikacji. Zweryfikowano klasy `DeepLieDetector` oraz `RFLieDetector` pod kątem zgodności interfejsów. Symulując wczytywanie wag z dysku, sprawdzono, czy metoda `predict` poprawnie przetwarza surowy tensor wejściowy i zwraca ustandaryzowany obiekt wynikowy zawierający prawdopodobieństwo, binarny werdykt oraz wektor wag uwagi (w przypadku Lasu Losowego, zwracane jest `None` jako wartość tego pola). Dzięki temu, potwierdzono, że warstwa interfejsu otrzymuje dane w oczekiwanym formacie niezależnie od wybranego modelu.

6.5 Prezentacja funkcjonalności i interfejsu użytkownika

6.5.1 Konfiguracja i panel sterowania

Interfejs został podzielony na dwa panele, co jest widoczne na poniższym rzucie ekranu. Z lewej strony znajduje się wysuwany panel konfiguracyjny, zawierający dynamiczny selektor modelu. Główny ekran służy do interakcji z danymi. Na jego górze znajduje się komponent do przesyłania plików. Kiedy użytkownik załaduje swoje nagranie, pojawia się jego nazwa pliku wraz z rozmiarem. Poniżej widnieje podgląd nagrania, dzięki któremu użytkownik może w łatwy sposób je obejrzeć w celu zweryfikowania czy analizuje właściwy materiał.



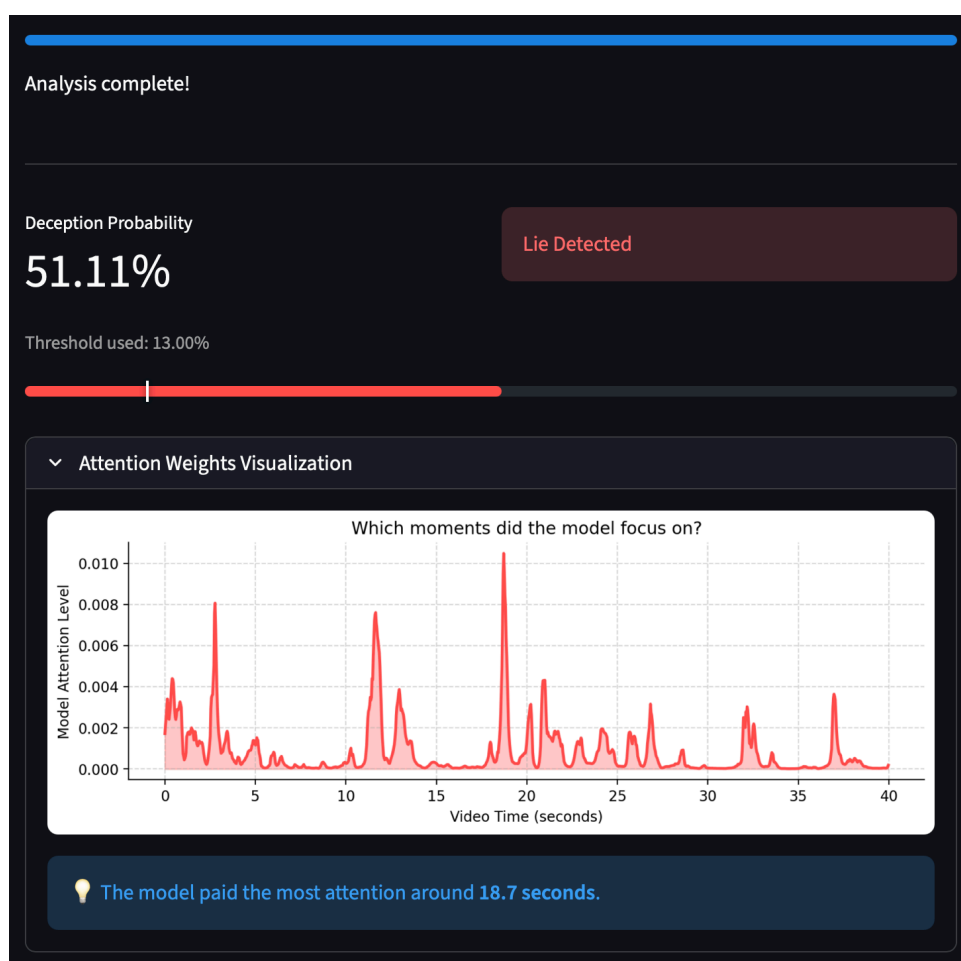
Rysunek 21. Widok główny aplikacji w fazie konfiguracji. W panelu bocznym widoczny wybór modelu, w części centralnej podgląd analizowanego nagrania.

6.5.2 Przebieg procesu inferencji

Wstępne przetworzenie wideo oraz wywołanie inferencji modelu jest dosyć długim procesem i w zależności od rozmiaru i długości nagrania może trwać nawet kilkadziesiąt sekund. Z tego powodu, zadbaneo żeby po kliknięciu przycisku „Analyze Video”, interfejs nie zamarał, lecz na bieżąco informował o aktualnie wykonywanych działaniach. W tym celu wykorzystano pasek postępu i komunikaty statusu, aby użytkownik nie zastanawiał się czy aplikacja się zawiesiła. Dodatkowo, w przypadku napotkania błędu (takiego jak np. niepomyślna detekcja twarzy na nagraniu) wyświetlany jest komunikat ostrzegawczy.

6.5.3 Prezentacja wyników

Na poniższym zrzucie ekranu przedstawiony został interfejs użytkownika wyświetlany po pomyślnie zakończonej inferencji. Wynik jest prezentowany w sposób binarny i probabilistyczny, co pozwala na natychmiastową interpretację wyniku przez użytkownika. Wyświetlony jest pasek pokazujący prawdopodobieństwo wystąpienia kłamstwa w stosunku do użytego progu decyzyjnego, w celu uzyskania maksymalnej transparentności. Na samym dole jako realizacja koncepcji wyjaśnialności sztucznej inteligencji (ang. *XAI - Explainable AI*) umieszczony został rozwijany panel wizualizujący wektor uwagi. Wykres ten pozwala zidentyfikować krytyczne momenty w nagraniu, które najmocniej wpłynęły na podjętą przez model decyzję. Ze względu na naturę alternatywnych modeli wykorzystujących algorytm lasu losowego, ten panel widoczny jest tylko po inferencji z użyciem modelu BiGRU+Attention.



Rysunek 22. Panel wyników analizy. Widoczna ocena prawdopodobieństwa kłamstwa, wizualizacja progu decyzyjnego oraz wykres wag atencji wskazujący na podejrzone fragmenty nagrania.

7 Podsumowanie

Celem niniejszej pracy było stworzenie systemu automatycznej detekcji kłamstwa, łączącego nowoczesne techniki wizji komputerowej i algorytmy sztucznej inteligencji z analizą behawioralną. Rozwiązanie problemu miało wysoce interdyscyplinarny charakter, o czym świadczy, że problem analizy szczerości wypowiedzi znajduje się na pograniczu psychologii, inżynierii danych i uczenia maszynowego. Jednomodalne podejście (analiza wyłącznie na podstawie klatek wideo) do implementacji takiego rozwiązania okazało się nietrywialnym zadaniem z powodu subtelności sygnałów oraz różnic w ekspresji emocjonalnej w zależności od wagi kłamstwa.

Poniższy rozdział, stanowiący zakończenie pracy, zawiera zestawienie zrealizowanych celów, podsumowanie wniosków uzyskanych z wyników eksperymentalnych, analizę ograniczeń napotkanych podczas realizacji projektu, ocenę praktycznej użyteczności stworzonego rozwiązania oraz propozycje na dalsze badania w tej dziedzinie.

7.1 Synteza zrealizowanych zadań

Udało się zaprojektować, zaimplementować i przetestować system automatycznie oceniający nagrania wideo pod kątem nieszczerości na podstawie analizy wizyjnej. Stworzono autorski potok przetwarzania wideo poczynając od surowego pliku, przez m.in. detekcję twarzy z użyciem modelu YOLO i ekstrakcję 478 punktów charakterystycznych twarzy, kończąc na inżynierii cech. Zaimplementowano i porównano dwa podejścia klasyfikacyjne:

- Klasyczne: algorytm Random Forest bazujący swoje predykcje na zagregowanych statystykach opisowych.
- Głębokie: autorski architektura BiGRU z mechanizmem uwagi, klasyfikujący nagrania na podstawie analizy sekwencyjnej.

Zaimplementowano działający interaktywny prototyp systemu w postaci aplikacji webowej, stanowiący użytkowe narzędzie wspomagające w procesie detekcji kłamstwa.

7.2 Kluczowe wnioski z badań

Przeprowadzone eksperymenty wykazały, że podczas oceny prawdopodobności wypowiedzi, ważniejsza niż jakość nagrań jest ich treść i kontekst emocjonalny. Zbiór **SDDD** zawiera nagrania w idealnej jakości technicznej, ale słaby sygnał kłamstwa (*low-stakes*). Nagrania ze zbioru **RLDDD**, mimo znacznie gorszej jakości, okazały się bardziej wartościowe dla modeli uczenia maszynowego. Potwierdza to wniosek, że waga kłamstwa wraz z silniejszym ładunkiem emocjonalnym była ważniejsza dla algorytmów klasyfikacyjnych niż sama rozdzielczość kamery lub studyjne warunki.

Badania potwierdziły również, że na małych zbiorach danych proste modele klasycznego uczenia maszynowego wygrywają ze skomplikowanymi architekturami, uzyskując lepszą dokładność predykcji oraz większą odporność na przeuczenie. Modele głębokie prawdopodobnie wymagają znacznie większego wolumenu danych, aby w pełni wykorzystać swój potencjał.

Zastosowanie techniki transferu wiedzy okazało się przełomowe dla skuteczności modelu głębokiego. Ewaluacja zero-shot wskazała, że model trenowany wyłącznie na danych laboratoryjnych z **SDDD** nie posiada zdolności generalizacji na dane rzeczywiste. Jednakże, po wymianie

warstwy klasyfikującej i wytrenowaniu jej na nowych danych, osiągnął on satysfakcjonujący wynik AUC na poziomie 0.76. Dowodzi to, że ekstraktor cech (BiGRU) nauczył się analizować dynamikę twarzy i mowę ciała, wymagał jedynie na nowo nauczyć się interpretować sygnały w nowym kontekście. To w połączeniu z wystąpieniem zjawiska katastroficznego zapominania świadczy o tym, że kłamstwo rzeczywiste i laboratoryjne to dwie zupełnie różne domeny, między którymi jest znaczna przepaść. Nie da się w prosty sposób zrobić jednego modelu do wszystkiego.

7.3 Ocena praktycznego zastosowania opracowanego systemu

Wdrożenie mechanizmu wizualizacji wag uwagi pozwoliło na realizację paradygmatu wyjaśnialnej sztucznej inteligencji (XAI), przekształcając system z „czarnej skrzynki” w transparentne i godne zaufania narzędzie analityczne. Jednak, biorąc pod uwagę uzyskane wyniki opisujące skuteczność wytrenowanych modeli, narzędzie to nie może być traktowane jako autonomiczny, bezbłędny detektor kłamstwa. Może być zastosowane, natomiast, do wspomagania człowieka w ręcznej analizie, wskazując momenty zawierające podejrzane zachowania i sugerujące stopień zaufania.

7.4 Proponowane kierunki rozwoju

Opracowane rozwiązanie stanowi solidną bazę do dalszych prac badawczo-rozwojowych. Najbardziej obiecującym kierunkiem rozwoju wydaje się wdrożenie analizy multimodalnej. Ścieżka audio dostarczyłaby informacji o tonie głosu i jego tempie, a transkrypcja wypowiedzi pozwoliłaby na dodatkową weryfikację wypowiedzianych zdań.

Głównym ograniczeniem napotkanym podczas realizacji projektu był rozmiar zbiorów danych. Zwiększenie bazy treningowej umożliwiłoby proponowanemu modelowi skuteczniejszy trening i lepszą generalizację, a także otworzyłoby drzwi na zastosowanie bardziej zaawansowanych architektur jak np. wcześniej opisana trójwymiarowa sieć konwolucyjna lub Transformer. Dodatkowo, zastosowanie dedykowanych sieci do wykrywania mikroekspresji zamiast analizy opartej wyłącznie o landmarki i przepływ optyczny z dużym prawdopodobieństwem zwiększyłoby czułość systemu na subtelne sygnały mimiczne.

Bibliografia

- [1] C. F. Bond Jr i B. M. DePaulo, „Accuracy of deception judgments”, *Personality and social psychology review*, t. 10, nr 3, s. 214–234, 2006.
- [2] P. Ekman i W. V. Friesen, „Nonverbal leakage and clues to deception”, *Psychiatry*, t. 32, nr 1, s. 88–106, 1969.
- [3] P. Ekman, *Telling lies: Clues to deceit in the marketplace, politics, and marriage*. WW Norton & Company, 2009.
- [4] J. Redmon, S. Divvala, R. Girshick, i A. Farhadi, „You only look once: Unified, real-time object detection”, w *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, s. 779–788.
- [5] S. Hochreiter i J. Schmidhuber, „Long short-term memory”, *Neural computation*, t. 9, nr 8, s. 1735–1780, 1997.
- [6] K. Radlak, M. Bozek, i B. Smolka, „Silesian Deception Database: Presentation and Analysis”, w *Proceedings of the 2015 ACM on Workshop on Multimodal Deception Detection*, w WMDD '15. Seattle, Washington, USA: Association for Computing Machinery, 2015, s. 29–35. doi: [10.1145/2823465.2823469](https://doi.org/10.1145/2823465.2823469).
- [7] V. Pérez-Rosas, M. Abouelenien, R. Mihalcea, i M. Burzo, „Deception Detection using Real-life Trial Data”, w *Proceedings of the 2015 ACM on International Conference on Multimodal Interaction*, w ICMI '15. Seattle, Washington, USA: Association for Computing Machinery, 2015, s. 59–66. doi: [10.1145/2818346.2820758](https://doi.org/10.1145/2818346.2820758).
- [8] A. Vrij, *Detecting lies and deceit: Pitfalls and opportunities*. John Wiley & Sons, 2008.
- [9] M. Zuckerman, B. M. DePaulo, i R. Rosenthal, „Verbal and nonverbal communication of deception”, *Advances in experimental social psychology*, t. 14, s. 1–59, 1981.
- [10] S. B. Porter, L. ten Brinke, i B. Wallace, „Secrets and Lies: Involuntary Leakage in Deceptive Facial Expressions as a Function of Emotional Intensity”, *Journal of Nonverbal Behavior*, t. 36, s. 23–37, 2011, [Online]. Dostępne na: <https://api.semanticscholar.org/CorpusID:28783661>
- [11] P. Ekman i W. V. Friesen, „Constants across cultures in the face and emotion”, *Journal of personality and social psychology*, t. 17, nr 2, s. 124–129, 1971.
- [12] P. Ekman, „An argument for basic emotions”, *Cognition and emotion*, t. 6, nr 3–4, s. 169–200, 1992.
- [13] P. Viola i M. J. Jones, „Robust real-time face detection”, *International journal of computer vision*, t. 57, nr 2, s. 137–154, 2004.
- [14] R. Coll Josifov, „Deep learning based computer vision for aerial-view street object detection and classification”, Doctoral dissertation, 2022. doi: [10.13140/RG.2.2.31481.54884](https://doi.org/10.13140/RG.2.2.31481.54884).
- [15] G. Jocher, A. Chaurasia, i J. Qiu, „Ultralytics YOLOv8”. GitHub, 2023.
- [16] C. Lugaresi *et al.*, „Mediapipe: A framework for building perception pipelines”, w *arXiv preprint arXiv:1906.08172*, 2019.

- [17] Y. Kartynnik, A. Ablavatski, I. Grishchenko, i M. Grundmann, „Real-time facial surface geometry from monocular video on mobile GPUs”, w *arXiv preprint arXiv:1907.06724*, 2019.
- [18] I. Grishchenko, A. Ablavatski, Y. Kartynnik, K. Rave, i M. Grundmann, „Attention mesh: High-fidelity face mesh prediction in real-time”, *arXiv preprint arXiv:2006.10962*, 2020.
- [19] B. K. Horn i B. G. Schunck, „Determining optical flow”, *Artificial intelligence*, t. 17, nr 1–3, s. 185–203, 1981.
- [20] G. Farneback, „Two-frame motion estimation based on polynomial expansion”, w *Proceedings of the 13th Scandinavian Conference on Image Analysis*, 2003, s. 363–370.
- [21] S.-T. Liong, R. C. Phan, J. See, Y.-H. Oh, i K. Wong, „Optical flow features for micro-expression recognition”, w *Proceedings of the 10th International Conference on Information Science and Applications*, 2014, s. 1–6.
- [22] T. Hastie, R. Tibshirani, i J. Friedman, *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2009.
- [23] L. Breiman, „Random forests”, *Machine learning*, t. 45, nr 1, s. 5–32, 2001.
- [24] I. Goodfellow, Y. Bengio, i A. Courville, *Deep learning*. MIT press, 2016.
- [25] Y. Bengio, P. Simard, i P. Frasconi, „Learning long-term dependencies with gradient descent is difficult”, *IEEE transactions on neural networks*, t. 5, nr 2, s. 157–166, 1994.
- [26] S. Hochreiter i J. Schmidhuber, „Long short-term memory”, *Neural computation*, t. 9, nr 8, s. 1735–1780, 1997.
- [27] K. Cho *et al.*, „Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”, w *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, s. 1724–1734.
- [28] M. Schuster i K. K. Paliwal, „Bidirectional recurrent neural networks”, *IEEE transactions on Signal Processing*, t. 45, nr 11, s. 2673–2681, 1997.
- [29] D. Bahdanau, K. Cho, i Y. Bengio, „Neural machine translation by jointly learning to align and translate”, *arXiv preprint arXiv:1409.0473*, 2014.
- [30] D. Tran, L. Bourdev, R. Fergus, L. Torresani, i M. Paluri, „Learning spatiotemporal features with 3d convolutional networks”, w *Proceedings of the IEEE international conference on computer vision*, 2015, s. 4489–4497.
- [31] T. Pfister, X. Li, G. Zhao, i M. Pietikäinen, „Recognising spontaneous facial micro-expressions”, w *2011 International Conference on Computer Vision*, 2011, s. 1449–1456.
- [32] V. Pérez-Rosas, M. Abouelenien, R. Mihalcea, Y. X. Burzo, i M. Burzo, „Deception detection using real-life trial data”, w *Proceedings of the 2015 ACM on International Conference on Multimodal Interaction*, 2015, s. 59–66.
- [33] M. Gogate, A. Adeel, i A. Hussain, „Deep learning for lie detection: Funneling macro and micro-expressions”, w *2017 IEEE International Joint Conference on Neural Networks (IJCNN)*, 2017, s. 3079–3085.

- [34] Q. Wu, X. Shen, i X. Fu, „Micro-expression recognition using robust principal component analysis and local spatiotemporal directional features”, *IEEE Transactions on Affective Computing*, t. 10, nr 4, s. 501–513, 2017.
- [35] Mattoss, „facial-emotion-recognition: A Python library for facial emotion recognition”. GitHub, 2020.
- [36] I. J. Goodfellow *et al.*, „Challenges in representation learning: A report on three machine learning contests”, w *Neural information processing*, 2013.

Wykaz symboli i skrótów

RLDDD – Real-Life Deception Detection Dataset: Zbiór danych zawierający nagrania wideo z prawdziwych sytuacji, w których osoby były nagrywane podczas mówienia prawdy lub kłamstwa. Nagrania pochodzą z różnych źródeł, takich jak programy telewizyjne, wywiady czy materiały z procesu sądowego. Są gorszej jakości niż nagrania z Silesian Deception Detection Dataset, lecz bardziej reprezentatywne dla rzeczywistych sytuacji.

SDDD – Silesian Deception Detection Dataset: Zbiór danych zawierający nagrania wideo przedstawiające osoby mówiące prawdę, bądź kłamiące. Nagrania powstały z użyciem profesjonalnej kamery w kontrolowanych warunkach laboratoryjnych na Politechnice Śląskiej.

Spis rysunków

Rysunek 1	Schemat architektury sieci YOLO. Źródło: [14]	6
Rysunek 2	Siatka punktów wygenerowana przez MediaPipe. Źródło: opracowanie własne korzystając z nagrania z SDDD	8
Rysunek 3	Wizualizacja gęstego przepływu optycznego. Po lewej: oryginalna klatka wideo (wykadrowana twarz). Po prawej: wizualizacja wektorów ruchu. Czarne tło oznacza brak istotnego ruchu (efekt zastosowanego progowania), natomiast barwne plamy wskazują na dynamikę w rejonie oczu, nosa i ust. Źródło: opracowanie własne z użyciem nagrania z SDDD.	9
Rysunek 4	Schemat sieci rekurencyjnej (RNN). Po lewej: reprezentacja zwinięta z pętlą sprzężenia zwrotnego. Po prawej: reprezentacja rozwinięta w czasie, ukazująca przepływ informacji (stanu ukrytego) przez kolejne kroki sekwencji. Źródło: opracowanie własne na podstawie [24].	11
Rysunek 5	Schemat komórki GRU (<i>Gated Recurrent Unit</i>). Widoczny przepływ sygnałów przez bramkę resetu (r_t) oraz bramkę aktualizacji (z_t). Symbol σ oznacza funkcję aktywacji sigmoidalną, a $\tanh$ tangens hiperboliczny. Źródło: opracowanie własne na podstawie [27].	13
Rysunek 6	Porównanie jakości próbek w wykorzystanych zbiorach danych. Po lewej: przykładowa klatka ze zbioru SDDD - widoczne idealne oświetlenie, jednolite tło i statyczna pozycja. Po prawej: przykładowa klatka ze zbioru RLDDD - niska rozdzielczość, małe zbliżenie i niejednolite tło.	17
Rysunek 7	Schemat blokowy potoku przetwarzania obrazu.	19
Rysunek 8	Schemat architektury modelu BiGRU z mechanizmem uwagi. Wymiary tensorów podano w nawiasach kwadratowych nad strzałkami przepływu danych.	25
Rysunek 9	Ważność cech: Zbiór RLDDD.	31
Rysunek 10	Ważność cech: Zbiór SDDD.	32
Rysunek 11	Krzywe uczenia uzyskane w procesie <i>Overfit Check</i>	33
Rysunek 12	Krzywe uczenia dla modelu BiGRU na zbiorze SDDD. Po lewej: przebieg funkcji straty (<i>BCEWithLogitsLoss</i>). Po prawej: przebieg metryki AUC ROC oraz F1 Score.	34
Rysunek 13	Przebieg optymalnego progu decyzyjnego.	35

Rysunek 14	Macierz pomyłek wytrenowanego modelu BiGRU+Attention na podzbiorze testowym zbioru SDDD	36
Rysunek 15	Histogram prawdopodobieństw predykcji modelu BiGRU+Attention na podzbiorze testowym zbioru SDDD	37
Rysunek 16	Statystyki uzyskane z ewaluacji zero-shot na zbiorze RLDDD . Po lewej: macierz pomyłek. Po prawej: metryki jakościowe.	38
Rysunek 17	Dynamika procesu transfer learningu modelu autorskiego na zbiorze RLDDD . W górnym rzędzie: przebieg funkcji straty (po lewej) oraz metryk walidacyjnych AUC i F1 (po prawej). U dołu: ewolucja optymalnego progu decyzyjnego w czasie trwania treningu.	39
Rysunek 18	Statystyki uzyskane z ewaluacji dotrenowanego modelu na zbiorze testowym z RLDDD . Po lewej: macierz pomyłek. Po prawej: metryki jakościowe.	40
Rysunek 19	Histogram prawdopodobieństw predykcji dotrenowanego modelu BiGRU+Attention na podzbiorze testowym zbioru RLDDD	41
Rysunek 20	Diagram klas UML prezentujący architekturę aplikacji. Widoczny podział na warstwę interfejsu (StreamlitApp), przetwarzania wideo (VideoProcessor) oraz hierarchię klas modułu predykcyjnego realizującą polimorfizm.	45
Rysunek 21	Widok główny aplikacji w fazie konfiguracji. W panelu bocznym widoczny wybór modelu, w części centralnej podgląd analizowanego nagrania.	47
Rysunek 22	Panel wyników analizy. Widoczna ocena prawdopodobieństwa kłamstwa, wizualizacja progu decyzyjnego oraz wykres wag atencji wskazujący na podejrzone fragmenty nagrania.	48

Spis tabel

Tabela 1	Zestawienie finalnych hiperparametrów modelu oraz procedury treningowej.	28
Tabela 2	Porównanie skuteczności modelu na zbiorach Silesian i Real Life	29
Tabela 3	Zestawienie wyników końcowych modelu autorskiego z modelem bazowym na zbiorze testowym Silesian.	36